

Electronics and Computer Science
Faculty of Engineering and Physical Sciences
University of Southampton

Virath Shuklla

9th December 2025

Code Feedback Studio

Project supervisor: Son Hoang
Second examiner: Tom Blount

A project report submitted for the award of
Beng. Software Engineering

Statement of Originality

- I have read and understood the [ECS Academic Integrity](#) information and the University's [Academic Integrity Guidance for Students](#).
- I am aware that failure to act in accordance with the [Regulations Governing Academic Integrity](#) may lead to the imposition of penalties which, for the most serious cases, may include termination of programme.
- I consent to the University copying and distributing any or all of my work in any form and using third parties (who may be based outside the EU/EEA) to verify whether my work contains plagiarised material, and for quality assurance purposes.

You must change the statements in the boxes if you do not agree with them.

We expect you to acknowledge all sources of information (e.g. ideas, algorithms, data) using citations. You must also put quotation marks around any sections of text that you have copied without paraphrasing. If any figures or tables have been taken or modified from another source, you must explain this in the caption and cite the original source.

I have acknowledged all sources, and identified any content taken from elsewhere.

If you have used any code (e.g. open-source code), reference designs, or similar resources that have been produced by anyone else, you must list them in the box below. In the report, you must explain what was used and how it relates to the work you have done.

I have not used any resources produced by anyone else.

You can consult with module teaching staff/demonstrators, but you should not show anyone else your work (this includes uploading your work to publicly-accessible repositories e.g. Github, unless expressly permitted by the module leader), or help them to do theirs. For individual assignments, we expect you to work on your own. For group assignments, we expect that you work only with your allocated group. You must get permission in writing from the module teaching staff before you seek outside assistance, e.g. a proofreading service, and declare it here.

I did all the work myself, or with my allocated group, and have not helped anyone else.

We expect that you have not fabricated, modified or distorted any data, evidence, references, experimental results, or other material used or presented in the report. You must clearly describe your experiments and how the results were obtained, and include all data, source code and/or designs (either in the report, or submitted as a separate file) so that your results could be reproduced.

The material in the report is genuine, and I have included all my data/code/designs.

We expect that you have not previously submitted any part of this work for another assessment. You must get permission in writing from the module teaching staff before re-using any of your previously submitted work for this assessment.

I have not submitted any part of this work for another assessment.

If your work involved research/studies (including surveys) on human participants, their cells or data, or on animals, you must have been granted ethical approval before the work was carried out, and any experiments must have followed these requirements. You must give details of this in the report, and list the ethical approval reference number(s) in the box below.

My work did not involve human participants, their cells or data, or animals.

ECS Statement of Originality Template, updated August 2018, Alex Weddell aiofficer@ecs.soton.ac.uk

Abstract

Code Feedback Studio is a web-based platform designed to improve the quality, clarity, and consistency of feedback in university programming courses. As class sizes increase, delivering timely and structured feedback becomes challenging for both students and markers. This project addresses these issues by providing a unified environment for code submission, manual review, moderation, and feedback delivery within an interactive browser-based interface.

The system enables markers to provide precise, line-by-line feedback directly on code, while supporting features such as reusable templates, cross-student comparison, and auto-draft saving to improve efficiency and consistency. Students benefit from structured, contextual feedback and iterative resubmissions, promoting deeper engagement with their learning. Gamification and reflection tools are also incorporated to support motivation and self-regulated learning.

The platform was evaluated through testing and user feedback, indicating improvements in usability and feedback clarity. However, limitations remain in terms of scalability testing and objective measurement of learning outcomes. Overall, Code Feedback Studio demonstrates a practical and user-centred approach to enhancing feedback processes in programming education.

Acknowledgements

I would like to express my sincere gratitude to Professor Son Hoang for his invaluable guidance and support throughout the development of this project. His expert advice, especially whenever I encountered doubts or challenges, has been instrumental in shaping the direction and success of the project. Professor Hoang's feedback and encouragement were crucial in refining both the technical and conceptual aspects of this report and the Code Feedback Studio platform.

I would also like to thank my peers, family, and friends for their constant encouragement and for providing constructive feedback during the various stages of the project. Their support kept me motivated and focused on delivering a project that I can be proud of.

Lastly, I extend my appreciation to the university for providing the resources and opportunities to undertake this project, and for fostering an environment where innovation in educational technology is encouraged.

Contents

Abstract.....	1
Acknowledgements	3
Chapter 1: Introduction.....	6
Chapter 2: Background and Literature Review	8
2.1. Background.....	8
2.2. Literature Review	8
2.2.1 Principles of Effective Feedback	8
2.2.2 Timeliness and Feedback Effectiveness	9
2.2.3 Challenges in Programming Assessment	9
2.2.4 Existing Tools and Their Limitations	9
2.2.5 Gamification and Student Engagement	10
2.2.6 Identified Research Gap	10
Chapter 3: Requirements and System Specification	12
3.1 Stakeholder Analysis	12
3.2 Personas and User Stories.....	13
3.3 Functional Requirements	15
3.4 Non-Functional Requirements	16
3.5 Product Backlog (MoSCoW).....	17
Chapter 4: Project Planning, Risk Assessment and Timeline.....	18
4.1 Risk Assessment	18
4.2 Project Planning and Timeline.....	20
Milestone 1 – Research and System Design Completion	20
Milestone 2 – Core Functionality Development.....	20
Milestone 3 – Feedback System and User Interface Development	21
Milestone 4 – Gamification and Testing	21
Milestone 5 – Finalisation and Documentation	21
Milestone 6 – Project Demonstration and Submission	21
4.3 Timeline Adaptation and Impact of Risks	21
Chapter 5: Design.....	23
5.1 Design Overview	23
5.2 User Interface Design	23
5.2.1 Student Interface.....	24
5.2.2 Marker Interface	27
5.3 Design Principles.....	27
Chapter 6: Implementation	29
6.1 Introduction to Implementation	29
6.2 Class Diagram.....	29
6.3 Package Architecture	31
6.4 Implementation Details.....	32
6.5 Challenges and Solutions.....	33
Chapter 7: Development Methodology and Tools	34
7.1 Agile Development Approach	34
7.2 Software Tools and Technologies.....	34
7.3 Frontend Technology Choices	34
7.4 Backend Technology Choices.....	35
7.5 Database and Data Management.....	35
7.6 Development and Deployment Tools.....	35
7.7 Benefits and Limitations of the Approach	36

7.8 Conclusion	36
Chapter 8: Testing and Evaluation	37
8.1 Overall Testing Approach.....	37
8.2 Backend Testing	37
8.3 Frontend Testing and User Interaction.....	38
8.4 Integration Testing (End-to-End Workflows).....	38
8.5 Data Validation and Edge Case Testing.....	39
8.6 Security Testing	39
8.7 Performance Evaluation.....	39
8.8 Regression Testing.....	40
8.9 Testing Results and Evidence	40
8.10 Critical Evaluation	40
8.11 User Feedback and Survey Analysis.....	41
Chapter 9: Reflection on Product Delivery	43
9.1. Project Scope and Requirement Realization	43
9.1.1 Implementation of Core Functionalities (The "Must-Haves")	43
9.1.2 Evolution of the Gamification Engine	43
9.2. Critical Evaluation of Data Policy and AI Integration	43
9.2.1 The University of Southampton Data Policy Compliance.....	43
9.3. Technical Summary and MoSCoW Finality	44
9.4 Critical Evaluation	44
Chapter 10: Conclusion and Lessons Learned	45
References	46

Chapter 1: Introduction

In university programming courses, feedback is essential for helping students improve their coding skills and understand key software development concepts. However, as class sizes increase, providing high-quality feedback becomes more difficult. Markers often have limited time to review a large number of submissions, which can lead to feedback that is brief, inconsistent, or delayed. At the same time, students may struggle to understand how feedback relates to their code, reducing its effectiveness as a learning tool.

Current approaches to marking programming assignments are often inefficient and fragmented. Feedback is typically provided through separate documents, PDFs, or basic learning platforms that do not support direct interaction with code. This makes it harder for students to link feedback to specific mistakes and for markers to maintain consistency across submissions. These limitations highlight the need for a more structured and integrated system that improves both the quality and delivery of feedback.

Code Feedback Studio is a web-based platform designed to address these issues. It provides a unified environment for code submission, manual review, moderation, and feedback delivery. Students can submit multi-file code assignments through the platform, while markers review submissions using an embedded code editor and provide precise, line-by-line feedback directly on the code. This ensures that feedback is clear, contextual, and easier to understand.

The system is implemented using a modern full-stack architecture. The frontend is built with React and Tailwind CSS, providing a responsive and user-friendly interface. The Monaco Editor is integrated to support code viewing with syntax highlighting and line-based interaction. The backend is developed using FastAPI, enabling efficient API handling, while MongoDB is used for flexible data storage. Features such as auto-draft saving ensure that marker input is not lost during the review process.

To improve efficiency and consistency, the platform includes reusable feedback templates and a cross-student comparison feature, allowing markers to review multiple submissions side by side. A moderation system is also implemented, where selected submissions are reviewed to ensure the quality and fairness of feedback.

For students, feedback is presented directly alongside their code, making it easier to identify and understand mistakes. The platform supports resubmissions, encouraging iterative improvement, and includes reflection tools that help students engage more actively with their learning process.

In addition, the platform incorporates gamification features such as points, badges, and levels to motivate consistent participation from both students and markers.

The main aim of this project is to design and implement a system that improves the quality, consistency, and effectiveness of feedback in programming education. The key objectives are:

- To develop a clear and user-friendly interface for both students and markers
- To enable structured, line-by-line feedback on code submissions
- To improve marking consistency through templates and comparison tools
- To implement moderation mechanisms to ensure feedback quality
- To support student learning through resubmissions and reflection features
- To increase engagement through gamification

- To evaluate the effectiveness of the system through testing

This report is structured to guide the reader progressively from the conceptual foundations of the project to its practical implementation and evaluation. Chapter 2 provides the academic background and literature review, establishing the theoretical basis for effective feedback in programming education and identifying the research gap addressed by this project. Chapter 3 translates these insights into system requirements, including stakeholder analysis, user stories, and functional specifications. Chapter 4 outlines the project planning, risk assessment, and timeline, explaining how the development process was managed and adapted. Chapter 5 presents the system design, focusing on user interface decisions and overall architecture, followed by Chapter 6, which details the technical implementation of the system. Chapter 7 discusses the development methodology and technology choices, including their benefits and trade-offs. Chapter 8 evaluates the system through testing and user feedback, critically analysing its performance and limitations. Finally, Chapters 9 and 10 provide reflection and concluding insights, highlighting key learnings, limitations, and directions for future improvement. This structure allows the reader to understand not only what was built, but also why it was built and how effectively it meets its intended objectives.

Chapter 2: Background and Literature Review

2.1. Background

Feedback is widely recognised as one of the most important factors influencing student learning, particularly in skill-based disciplines such as programming. In university programming courses, students are expected not only to produce correct code but also to develop good practices in logic, structure, readability, and efficiency. Effective feedback helps students understand mistakes, refine their thinking, and improve future performance.

However, delivering high-quality feedback at scale remains a significant challenge. As class sizes grow, markers must review a large number of submissions within limited time. This often results in feedback that is brief, delayed, or inconsistent. Students, in turn, may struggle to understand how feedback relates to their code, especially when it is provided in a separate format such as a document or PDF. These issues reduce the effectiveness of feedback and limit opportunities for meaningful learning.

Traditional tools used in programming education, such as learning management systems, do not fully support the needs of code-based assessment. They often lack direct interaction with code, structured feedback mechanisms, and support for consistency across markers. As a result, there is a clear need for a system that integrates submission, review, and feedback into a single, structured environment.

This project, Code Feedback Studio, is developed to address these challenges by focusing on clarity, structure, consistency, and engagement in the feedback process.

2.2. Literature Review

2.2.1 Principles of Effective Feedback

Research by [Nicol and Macfarlane-Dick \(2006\) \[1\]](#) identifies key principles of effective feedback, including:

- feedback should be clear and understandable
- it should be timely
- it should help students reflect and self-correct

Similarly, [Hattie and Timperley \(2007\) \[2\]](#) argue that feedback is most effective when it answers three questions:

1. What is the goal?
2. How is the student performing?
3. What should be done next?

How this informed the project:

These principles directly influenced the design of Code Feedback Studio:

- Line-by-line feedback ensures clarity by linking comments to exact code locations
- Immediate visibility of feedback improves timeliness
- Reflection features (pre- and post-feedback) support self-regulated learning

- Structured issue format (title, explanation, fix) ensures feedback answers “what went wrong” and “what to do next”

2.2.2 Timeliness and Feedback Effectiveness

[Gibbs and Simpson \(2004\) \[3\]](#) highlight that delayed feedback significantly reduces its value. If students receive feedback too late, they are less likely to use it for improvement because they have already moved on to new topics.

How this informed the project:

To address this issue:

- the system supports efficient marking workflows through templates and structured inputs
- auto-draft saving reduces time lost during marking
- resubmission support (if the module leader wants to) allows students to immediately act on feedback

This ensures feedback remains relevant and actionable.

2.2.3 Challenges in Programming Assessment

Programming assessment presents unique challenges. According to [Ihantola et al. \(2010\) \[4\]](#), reviewing programming assignments requires significant manual effort, especially when feedback is detailed. In addition, programming feedback must consider:

- logic and correctness
- code structure and readability
- efficiency and design

Another key issue is that feedback is often detached from the code, making it harder for students to understand.

How this informed the project:

This research led to several core design decisions:

- use of a code editor (Monaco Editor) to provide feedback directly on code
- support for multi-file submissions, reflecting real programming tasks
- structured issue categories (e.g., logic, style, efficiency)
- inline highlighting to remove the disconnect between feedback and code

These features ensure that feedback is both contextual and meaningful.

2.2.4 Existing Tools and Their Limitations

Common platforms such as [Moodle \[6\]](#) and [Blackboard \[7\]](#) provide basic assignment submission and grading functionality. However, they lack:

- detailed code interaction
- structured feedback systems
- tools for maintaining consistency across markers

More advanced platforms like GitHub Classroom and [CodeGrade \[8\]](#) support inline commenting and code-based workflows. While these tools improve feedback delivery, they are often:

- designed for professional development environments
- less focused on educational workflows
- lacking features such as moderation and structured templates

How this informed the project:

Code Feedback Studio builds on these systems but adapts them for education:

- simplified, student-friendly interface
- feedback templates for consistency
- moderation pipeline for quality control
- comparison feature to support fair marking

This positions the platform as an education-focused alternative, rather than a generic code review tool.

2.2.5 Gamification and Student Engagement

[Deterding et al. \(2011\) \[5\]](#) define gamification as the use of game design elements, such as points and badges, in non-game contexts. Research shows that gamification can:

- increase motivation
- encourage participation
- reinforce positive behaviours

However, it must be carefully implemented to support learning rather than distract from it.

How this informed the project:

Gamification in Code Feedback Studio is designed to:

- reward productive behaviours (e.g., fixing issues, submitting work)
- encourage continuous improvement
- provide visual progress tracking (XP, levels, badges)

Importantly, it does not replace learning objectives but supports them.

2.2.6 Identified Research Gap

From the literature, it is clear that:

- feedback is essential but often poorly delivered
- programming assessment is complex and time-consuming
- existing tools provide partial solutions but lack integration

There is a gap in systems that combine:

- structured, line-by-line feedback
- marking consistency tools

- moderation mechanisms
- student engagement features

How this informed the project:

Code Feedback Studio is designed specifically to fill this gap by integrating:

- submission + review + feedback in one system
- consistency tools (templates, comparison)
- quality control (moderation pipeline)
- engagement features (gamification, reflections)

Chapter 3: Requirements and System Specification

3.1 Stakeholder Analysis

Stakeholder	Stakeholder type	Role/description
Students (Learners)	Primary	Use the system to view feedback on their code, understand mistakes, track progress, and improve their coding skills through clear explanations and guided suggestions.
Markers (Instructors, TAs, Tutors)	Primary	Markers use the platform to review student code, add detailed inline comments, create and apply reusable feedback templates, and compare different students' submissions side by side when needed. They can then publish clear and structured final feedback. Their interaction with the system plays a crucial role in shaping the quality, consistency, and effectiveness of student learning.
Module Leaders / Course Coordinators	Primary	Oversee the marking process and ensure consistency across markers. Use system analytics to identify common student issues and improve module design and teaching materials.
University Administrators / Education Quality Team	Secondary	Interested in how the platform improves teaching quality, reduces marking time, and increases consistency in feedback across large modules and cohorts.
Software Developers / System Maintainers	Secondary	Responsible for maintaining, updating, and improving the Code Feedback Studio platform. Ensure system reliability, security, and integration with university systems.
Vendors & Third-Party Integration Providers	Tertiary	Provide tools or APIs (e.g., plagiarism detection, version control integration, cloud hosting) that the system relies on.
Ethics & Data Protection Officers (GDPR/University Compliance)	Tertiary	Ensure the system handles data correctly, protects student privacy, and follows legal requirements for use and data storage.

The stakeholder analysis identifies the key groups involved in or affected by the system and clarifies their roles, expectations, and level of influence. By categorising stakeholders into primary, secondary, and tertiary groups, the analysis highlights which users have the most direct interaction with the system and

whose needs must be prioritised during development. Primary stakeholders, such as students, markers, and module leaders, directly shape the system’s functionality, while secondary and tertiary stakeholders influence aspects such as system maintenance, scalability, and compliance. This structured understanding ensures that the platform is designed not only to support end users but also to meet institutional, technical, and regulatory requirements.

3.2 Personas and User Stories

Stakeholder	Persona	Persona Description	User Story	Epic
Student	Aisha Khan	Aisha is a 20-year-old second-year Computer Science student. She enjoys programming but often struggles to understand the feedback she receives on her assignments. Many of her markers write feedback in different styles, and some comments are too technical or vague. She wants clearer explanations and examples so she can learn how to improve her code. Aisha also prefers visual feedback and interactive tools because they make learning easier. She is motivated by progress tracking and enjoys earning badges and achievements in apps.	As a student, I want to view feedback directly on my code so that I can clearly understand which part is wrong. <i>(Epic 1)</i>	1
			As a student, I want explanations in simple language so that I can understand the feedback without feeling confused. <i>(Epic 1)</i>	1
			As a student, I want to see “What went wrong”, “Why it matters”, and “How to fix it” for each comment so that I know how to improve. <i>(Epic 1)</i>	1
			As a student, I want to resubmit improved code and track whether my mistakes have been fixed so that I can measure my progress. <i>(Epic 2)</i>	2
			As a student, I want to earn points and badges when I fix mistakes so that learning feels more motivating. <i>(Epic 3)</i>	3
Marker / TA / Instructor	Dr. James Turner	Dr. Turner is a 34-year-old teaching assistant who handles marking for large programming classes. He often has hundreds of submissions to review, which makes it difficult to give detailed, consistent feedback to every student. He is comfortable with code but finds it time-consuming to type similar comments repeatedly. He wishes he had tools that	As a marker, I want to open a student’s code in a code viewer so that I can easily navigate through files. <i>(Epic 1)</i>	1
			As a marker, I want to add inline comments to the exact lines of code so that students know exactly where the issue is. <i>(Epic 1)</i>	1
			As a marker, I want to be able to create reusable feedback templates so that I can provide consistent and efficient feedback across multiple submissions. <i>(Epic 3)</i>	3

		could help him quickly spot mistakes, write clearer feedback, and keep marking consistent across different TAs.	As a marker, I want to see analytics of common errors across the class so that I know which topics students struggle with. <i>(Epic 4)</i>	4
Module Leader	Professor Emily Hughes	Professor Hughes is 47 and leads first- and second-year programming modules. She manages multiple TAs and wants all feedback to be fair, consistent, and aligned with the module rubric. She also needs insights into which topics students struggle with so she can update lectures and assignment designs. She values reliability, academic integrity, and transparency in the marking process.	As a module leader, I want to ensure all markers follow the rubric so that feedback is consistent across the module. <i>(Epic 1)</i>	1
			As a module leader, I want to review class-wide analytics so that I can identify weak areas in student understanding. <i>(Epic 2)</i>	2
System Developer / IT Maintainer	Leo Martinez	Leo is a 28-year-old full-stack developer who maintains university systems. He focuses on keeping platforms stable, secure, and easy to integrate with tools like SSO. He values clear documentation, strong security, and scalable systems that require minimal manual maintenance.	As a system developer, I want the platform to integrate with existing university login systems so that students can sign in easily. <i>(Epic 4)</i>	4
			As a system developer, I want all client data to be stored securely so that privacy is protected. <i>(Epic 4)</i>	4
			As a system developer, I want clear logs and error messages so that I can diagnose issues quickly. <i>(Epic 5)</i>	5

The personas and user stories build on the stakeholder analysis by translating general user needs into specific, actionable requirements. By creating realistic user profiles, such as students struggling with unclear feedback or markers managing large workloads, the system design is grounded in real-world challenges rather than abstract assumptions. The user stories clearly define what each stakeholder wants to achieve and why, helping guide feature development in a user-centred manner. Organising these stories into epics further supports prioritisation and ensures that related functionalities are grouped logically. This approach helps ensure that the final system is aligned with user expectations and effectively addresses the core problems identified in the project.

3.3 Functional Requirements

Functional requirements define the core capabilities of the system and ensure that stakeholder needs are translated into implementable features. Each requirement is linked to a concrete technical implementation, ensuring traceability between system design and development.

ID	Requirement	Description	Implementation
FR1	User Registration & Login	Users can register and log in securely with email and password	Implemented using JWT authentication and bcrypt password hashing
FR2	Role-Based Access Control	Different roles (student, marker, moderator, leader) have different permissions	Enforced in backend using role checks (RBAC)
FR3	Course Management	Markers can create courses and manage students	Implemented via course APIs and database collections
FR4	Assignment Creation	Assignments can be created with deadlines and marks	Backend assignment model with configurable properties
FR5	Code Submission	Students can submit multi-file code	Implemented using Monaco Editor and submission APIs
FR6	Inline Feedback	Markers can add feedback on specific lines of code	Implemented using FeedbackIssue model and line selection
FR7	Feedback Templates	Markers can reuse common feedback	Stored in database and auto-filled in UI
FR8	Grading System	Markers can assign marks and comments	Implemented via grading API with submission updates
FR9	Feedback Publishing	Feedback can be released to students	Submission status updated to "feedback_released"
FR10	Moderation System	Feedback can be reviewed and approved	Moderation queue with issue tracking
FR11	Cross-Student Comparison	Compare two submissions side-by-side	Dual Monaco editor interface
FR12	Student Reflections	Students can write pre/post reflections	Stored in reflection collection with prompts
FR13	Gamification	Users earn XP, badges, and levels	Backend gamification logic and tracking system
FR14	Leaderboard	Users can view rankings per module	Leaderboard API with scoring logic
FR15	PDF Export	Feedback can be exported as PDF	ReportLab-based PDF generation service

FR16	Auto-Draft Saving	Work is automatically saved during marking	Periodic backend draft-saving system
-------------	-------------------	--	--------------------------------------

These requirements demonstrate how user needs were translated into system functionality. The inclusion of implementation details strengthens the link between design and execution, enabling validation of system completeness and correctness.

3.4 Non-Functional Requirements

Non-functional requirements define the quality attributes that determine how effectively the system operates in practice. These are critical in ensuring that the system is secure, reliable, and scalable in a real-world academic environment.

ID	Requirement	Description	Implementation
NFR1	Security	Protect user data and prevent unauthorised access	JWT authentication, bcrypt hashing, RBAC
NFR2	Performance	System should respond quickly	Async FastAPI with Motor (MongoDB driver)
NFR3	Scalability	Support multiple users simultaneously	Stateless APIs and asynchronous operations
NFR4	Usability	System should be easy to use	React UI with Tailwind CSS and structured design
NFR5	Reliability	Prevent data loss during usage	Auto-draft saving and recovery mechanisms
NFR6	Maintainability	Code should be easy to update	Modular architecture and layered design
NFR7	Compatibility	Work across modern browsers	Tested on Chrome, Firefox, Edge
NFR8	Data Integrity	Ensure consistency of stored data	Validations using Pydantic models
NFR9	Availability	System should remain accessible	Cloud deployment readiness (Vercel + Render)
NFR10	Extensibility	System should support future features	Flexible MongoDB schema and modular backend

Security and data integrity are particularly important due to the handling of student submissions and academic records. These requirements ensure that the system is not only functional but also robust and suitable for institutional deployment.

3.5 Product Backlog (MoSCoW)

Must:

1. The system must allow markers to upload or access student code submissions.
2. The system must display student code in an interactive code viewer.
3. The system must allow markers to add inline comments to specific lines of code.
4. The system must allow students to view feedback directly on their code.
5. The system must support student resubmissions and show updated feedback.
6. The system must store user data, submissions, and feedback securely.
7. The system must encrypt sensitive information such as passwords.
8. The system must provide clear error messages when an action fails.
9. The system must display a loading indicator while processing or retrieving data.
10. The system must allow authenticated access for students and markers using role-based permissions.

Should:

1. The system should provide a simple progress overview for each student.
2. The system should include minimal gamification, such as an XP bar or progress percentage.
3. The system should allow markers to filter submissions by status (e.g., pending, completed).
4. The system should allow students to view previous submissions and feedback history.
5. The system should allow export of feedback as a downloadable PDF.
6. The system should allow module leaders to access basic analytics (e.g., submission counts, etc)
7. The system should allow markers to reuse predefined feedback templates.

Could:

1. The system could offer accessibility themes such as dark mode or high-contrast mode.
2. The system could allow students to add personal notes or reflections to their submissions.
3. The system could provide simple charts showing progress trends over time.
4. The system could allow side-by-side comparison of original and resubmitted code.
5. The system could include badge-style milestones for student motivation.
6. The system will include advanced gamification such as leaderboards.

Won't:

1. The system will not automatically grade or test student code.

Chapter 4: Project Planning, Risk Assessment and Timeline

4.1 Risk Assessment

Risk ID	Risk Description	Likelihood	Impact	Mitigation Strategy	Contingency Plan (What I will do if it fails)
R2	Monaco Editor integration issues or unexpected bugs	Low	Medium	Follow official docs and test integration early	Replace Monaco with CodeMirror or Ace Editor
R3	FastAPI backend errors, crashes, or high latency	Medium	High	Add logging, load tests, optimise endpoints	Switch temporarily to Flask or simplify slow features
R4	MongoDB connection failures, schema issues, or database corruption	Low	High	Regular backups; use Alembic migrations	Use SQLite temporarily until PostgreSQL is restored
R5	GitHub Actions CI/CD pipeline failure	Medium	Medium	Keep workflow simple; test locally before pushing	Deploy manually using Azure portal
R6	Azure App Service student tier credits expire or deployment errors	Medium	High	Monitor with Azure logs; keep deployments lightweight	Move to free hosts like Render, Railway, or Vercel
R8	Students or markers find the UI confusing	Medium	Medium	Conduct early UI testing; gather user feedback	Redesign UI flow based on actual user issues
R9	Time constraints cause incomplete features	Medium	High	Use MoSCoW prioritisation	Drop non-essential features (gamification, analytics)
R10	Syntax or style analysis fails for certain inputs	Medium	Medium	Use reliable libraries (pylint/flake8); sanitise inputs	Temporarily disable automated checking and rely on human review
R11	Version control conflicts cause lost work	Medium	High	Use feature branches; commit often	Revert to stable commits and reapply changes manually
R12	Laptop or hardware failure	Low	High	Store all code on GitHub	Use university computers; clone repo to any machine

R13	Internet outage affects development or deployment	Medium	Medium	Keep full local dev setup working offline	Work fully locally until internet returns
R14	Supervisor delays or unavailability	Medium	Medium	Plan questions early; send summaries	Adjust milestones; get peer or online guidance meanwhile
R15	Scope creep from adding too many features	Medium	High	Stick to defined scope; maintain separate backlog	Freeze new ideas and focus on core features
R16	Free-tier tools cause rate limits or slow performance	Medium	Medium	Monitor usage; optimise requests	Reduce feature load or move to alternative free tier
R17	Data privacy concerns handling student code	Low	High	Use mock/test code; avoid personal data	Remove any sensitive data and follow ethics guidelines
R19	Bugs take longer to fix than estimated	High	Medium	Break tasks into smaller units; write tests	Delay enhancements and focus on core functionality
R20	Figma or design tools restrict features on free tier	Medium	Low	Keep designs simple; use screenshots instead of exports	Switch to draw.io or basic offline diagramming tools
R21	Browser compatibility issues (Safari/Firefox differences)	Medium	Medium	Test UI on multiple browsers early	Add fallback UI or restrict unsupported browsers temporarily
R22	Security vulnerabilities (unsafe endpoints, missing validation)	Low	High	Validate all inputs; use FastAPI security practices	Disable vulnerable endpoints and patch immediately
R24	Free-tier hosting becomes too slow during testing	Medium	Medium	Optimise bundle size and remove unused scripts	Move frontend to Vercel and backend to Render (both free)
R25	Limited number of real users for testing	Medium	Medium	Recruit peers for testing; simulate user flows	Use generated test data or scripted scenarios

4.2 Project Planning and Timeline

Task	Phase	Start Date	End Date	Duration (days)	Progress
Project Research and Planning	Milestone 0 – Research	08/Oct/2025	14/Nov/2025	37	0%
Background Study	Milestone 0 – Research	08/Oct/2025	21/Oct/2025	13	0%
Literature Review	Milestone 0 – Research	22/Oct/2025	05/Nov/2025	14	0%
Requirements and Scope Definition	Milestone 0 – Research	06/Nov/2025	14/Nov/2025	8	0%
System Design and Architecture	Milestone 1 – Design	22/Nov/2025	01/Dec/2025	9	0%
Milestone 1	Milestone 1 – Design	22/Nov/2025	12/Dec/2025	20	0%
Initial Design Phase	Milestone 1 – Design	01/Dec/2025	05/Dec/2025	4	0%
System Architecture and Database Planning	Milestone 1 – Design	06/Dec/2025	12/Dec/2025	6	0%
Milestone 2	Milestone 2 – Core	23/Dec/2025	06/Mar/2026	73	0%
Increment 1 – Core Functionality Development	Milestone 2 – Core	23/Dec/2025	30/Jan/2026	38	0%
Code Submission and Storage Features	Milestone 2 – Core	04/Feb/2026	20/Feb/2026	16	0%
Increment 2 – Feedback Engine Development	Milestone 2 – Feedback	20/Feb/2026	06/Mar/2026	14	0%
Milestone 3	Milestone 3 – Feedback	06/Mar/2026	01/Apr/2026	26	0%
Feedback Generation and Integration	Milestone 3 – Feedback	06/Mar/2026	10/Mar/2026	4	0%
User Interface for Feedback Display	Milestone 3 – Feedback	10/Mar/2026	16/Mar/2026	6	0%
Increment 3 – User Interaction and Gamification	Milestone 3 – UI & Gamification	16/Mar/2026	01/Apr/2026	16	0%
Milestone 4	Milestone 4 – UI & Gamification	01/Apr/2026	15/Apr/2026	14	0%
Gamification Elements (Badges and XP)	Milestone 4 – UI & Gamification	01/Apr/2026	10/Apr/2026	9	0%
Increment 4 – Testing and Refinement	Milestone 4 – Testing	11/Apr/2026	13/Apr/2026	2	0%
System Integration and Functional Testing	Milestone 4 – Testing	13/Apr/2026	15/Apr/2026	2	0%
Milestone 5	Milestone 5 – Testing	15/Apr/2026	25/May/2026	40	0%
Bug Fixes and Improvements	Milestone 5 – Testing	15/Apr/2026	16/Apr/2026	1	0%
Documentation and Final Submission	Milestone 5 – Docs	16/Apr/2026	28/Apr/2026	12	0%
Final Report Writing	Milestone 5 – Docs	16/Apr/2026	28/Apr/2026	12	0%
Editing and Proofreading	Milestone 5 – Docs	22/Apr/2026	28/Apr/2026	6	0%

Figure 4.1: Initial Project Timeline (Planned Gantt Chart)

Task	Phase	Start Date	End Date	Duration (days)	Progress
Project Research and Planning	Milestone 0 – Research	08/Oct/2025	16/Nov/2025	39	100%
Background Study	Milestone 0 – Research	08-Oct-25	22-Oct-25	14	100%
Literature Review	Milestone 0 – Research	20-Oct-25	08-Nov-25	19	100%
Requirements and Scope Definition	Milestone 0 – Research	05-Nov-25	16-Nov-25	11	100%
System Design and Architecture	Milestone 1 – Design	22-Nov-25	08-Dec-25	16	100%
Milestone 1	Milestone 1 – Design	22/Nov/2025	15/Dec/2025	23	100%
Initial Design Phase	Milestone 1 – Design	01-Dec-25	10-Dec-25	9	100%
System Architecture and Database Planning	Milestone 1 – Design	08-Dec-25	15-Dec-25	7	100%
Milestone 2	Milestone 2 – Core	15/Dec/2025	15/Mar/2026	90	100%
Increment 1 – Core Functionality Development	Milestone 2 – Core	15-Dec-25	05-Feb-26	52	100%
Code Submission and Storage Features	Milestone 2 – Core	25-Jan-26	20-Feb-26	26	100%
Increment 2 – Feedback Engine Development	Milestone 2 – Feedback	15-Feb-26	15-Mar-26	28	100%
Milestone 3	Milestone 3 – Feedback	10/Mar/2026	05/Apr/2026	26	100%
Feedback Generation and Integration	Milestone 3 – Feedback	10-Mar-26	20-Mar-26	10	100%
User Interface for Feedback Display	Milestone 3 – Feedback	15-Mar-26	25-Mar-26	10	100%
Increment 3 – User Interaction and Gamification	Milestone 3 – UI & Gamification	20-Mar-26	05-Apr-26	16	100%
Milestone 4	Milestone 4 – UI & Gamification	01/Apr/2026	22/Apr/2026	21	100%
Gamification Elements (Badges and XP)	Milestone 4 – UI & Gamification	01-Apr-26	12-Apr-26	11	100%
Increment 4 – Testing and Refinement	Milestone 4 – Testing	08-Apr-26	20-Apr-26	12	100%
System Integration and Functional Testing	Milestone 4 – Testing	12-Apr-26	22-Apr-26	10	100%
Milestone 5	Milestone 5 – Testing	15/Apr/2026	28/Apr/2026	13	100%
Bug Fixes and Improvements	Milestone 5 – Testing	15-Apr-26	22-Apr-26	7	100%
Final Report Writing	Milestone 5 – Docs	16-Apr-26	26-Apr-26	10	100%
Documentation and Final Submission	Milestone 5 – Docs	18-Apr-26	28-Apr-26	10	100%
Editing and Proofreading	Milestone 5 – Docs	24-Apr-26	28-Apr-26	4	100%

Figure 4.2: Final Project Timeline (Revised Gantt Chart After Implementation)

Milestone 1 – Research and System Design Completion

By the end of this milestone, all the initial research and planning work was completed. This included background study, reviewing existing systems, and defining the requirements of the project. I also worked on the system design and overall structure of the platform. This stage helped build a clear understanding of what the system needed to do and how it would work before starting development.

Milestone 2 – Core Functionality Development

In this stage, the main features of Code Feedback Studio were developed. This included allowing students to submit code and enabling markers to access and review those submissions. The basic feedback system was also built, where markers could add comments directly to the code. This milestone created the core working version of the platform.

Milestone 3 – Feedback System and User Interface Development

During this phase, I focused on improving how feedback is shown to students. A clean and easy-to-use interface was developed so that students could clearly understand the comments given by markers. I also worked on improving user interaction and began adding early engagement features. By the end of this stage, the system became much more user-friendly.

Milestone 4 – Gamification and Testing

In this milestone, engagement features like badges and experience points (XP) were added to make the platform more interactive and motivating for students. At the same time, I carried out testing to identify and fix issues. All parts of the system were brought together and checked to make sure everything worked smoothly.

Milestone 5 – Finalisation and Documentation

This stage focused on final improvements. Bugs were fixed, performance was improved, and the system was made more stable. Alongside this, I completed the project report, documentation, and final edits. By the end of this milestone, both the system and the report were ready for submission.

Milestone 6 – Project Demonstration and Submission

The final milestone involved preparing the project for submission and demonstration. I ensured that everything was complete, working properly, and ready to be presented. This marked the completion of the project.

4.3 Timeline Adaptation and Impact of Risks

The project timeline evolved significantly from the initial plan, reflecting both technical challenges and risks encountered during development. While the original Gantt chart (Figure 4.1) followed a structured and largely sequential workflow, the final timeline (Figure 4.2) became more iterative and overlapping, requiring continuous adaptation, reprioritisation, and parallel task execution.

Several risks directly influenced these changes. The shift from in-person to online supervisor meetings reduced the immediacy of feedback, leading to slower clarification of design decisions and delays in progressing certain components. Hosting constraints also emerged when Microsoft Azure student-tier credits were exhausted, requiring migration to Vercel (frontend) and Render (backend). This introduced additional time for redeployment, configuration, and testing. Hardware limitations further impacted progress, as a laptop failure temporarily halted development and disrupted continuity. Additionally, debugging complex issues—particularly those involving backend logic and system integration—consistently took longer than expected, highlighting the unpredictability of development timelines.

Milestone 2 (Core Development) was extended due to the complexity of implementing core features such as submission handling and backend architecture. Strong interdependencies between components required additional refinement, causing overlap with subsequent phases, particularly the feedback engine. As a result, development tasks in Increment 1 and Increment 2 were extended and partially parallelised to maintain progress.

In Milestone 3, user interface development was expanded beyond its initial schedule to accommodate multiple design iterations aimed at improving clarity and usability, especially in presenting feedback alongside code. Similarly, Milestone 4 (Testing and Refinement) evolved from a short, final-stage activity into a longer and continuous process, as integration issues required repeated testing across workflows such as submission, feedback, and moderation.

Finally, Milestone 5 (Finalisation and Documentation) was restructured into a highly parallel phase to meet the submission deadline. Tasks such as bug fixing, report writing, and documentation were carried out simultaneously, reflecting significant time pressure in the final stages of the project.

Overall, the differences between the planned and final timelines highlight several key challenges:

- Underestimation of the complexity of core system implementation
- Strong interdependencies between frontend and backend components
- Increased time required for debugging, integration, and testing
- External disruptions, including hosting limitations and hardware failure
- Reduced feedback cycles due to changes in supervisor availability
- Time pressure near submission requiring parallel task execution

This experience demonstrates the importance of adaptive project management in software development. While the initial timeline provided a structured foundation, it became evident that flexibility, iterative development, and continuous reprioritisation were essential for successful delivery. It reinforces that real-world software projects rarely follow linear plans and require the ability to adapt effectively to both technical and external challenges.

Chapter 5: Design

5.1 Design Overview

The design of Code Feedback Studio focuses on creating a clean, intuitive interface for two primary user groups: students and markers. As their workflows differ, the system was designed to support distinct tasks while maintaining a consistent visual style. Students require a simple way to view feedback and track progress, whereas markers need efficient tools for reviewing code and managing submissions.

The design is guided by key Human-Computer Interaction principles such as consistency, visibility of system status, and clear feedback ([Nielsen, 1994 \[13\]](#)). These principles are reflected in the use of card-based layouts, clear spacing, and readable typography, all aimed at reducing cognitive load. The inclusion of dark mode further enhances usability by improving visual comfort during extended usage ([Budiu, 2020 \[19\]](#)). To avoid overwhelming users, simplicity was prioritised over feature density, ensuring the platform remains focused on feedback and marking tasks.

5.2 User Interface Design

The interface is structured around clarity, accessibility, and workflow efficiency. A consistent visual language is maintained through simple layouts, clear spacing, and intuitive navigation, allowing users to move easily between different sections.

Status indicators provide immediate feedback on submission progress, improving system visibility. The use of progressive disclosure ensures that essential information is presented first, with additional details available when needed, reducing complexity ([Tidwell, 2010 \[18\]](#)).

While this approach improves usability, it may limit access to advanced features. Future iterations could introduce optional controls for experienced users without compromising the simplicity of the core interface.

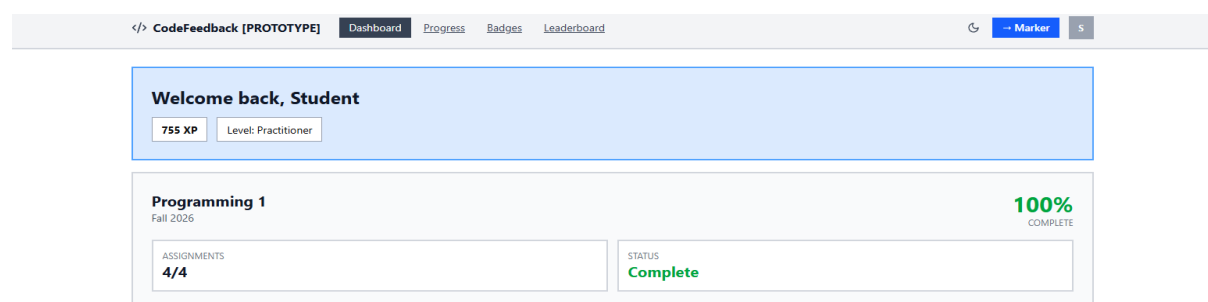


Figure 5.1: Student Dashboard Interface Showing Course Progress and Completion Status

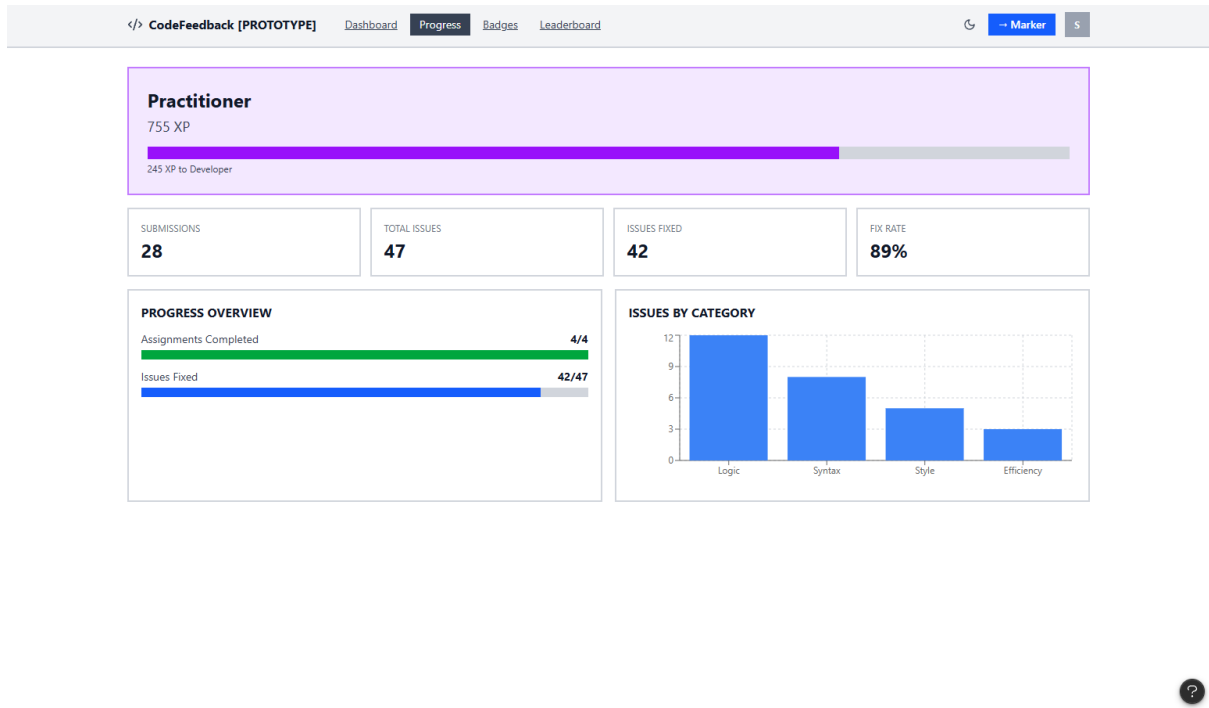


Figure 5.2: Student Progress Dashboard with XP Tracking, Performance Metrics, and Issue Analysis

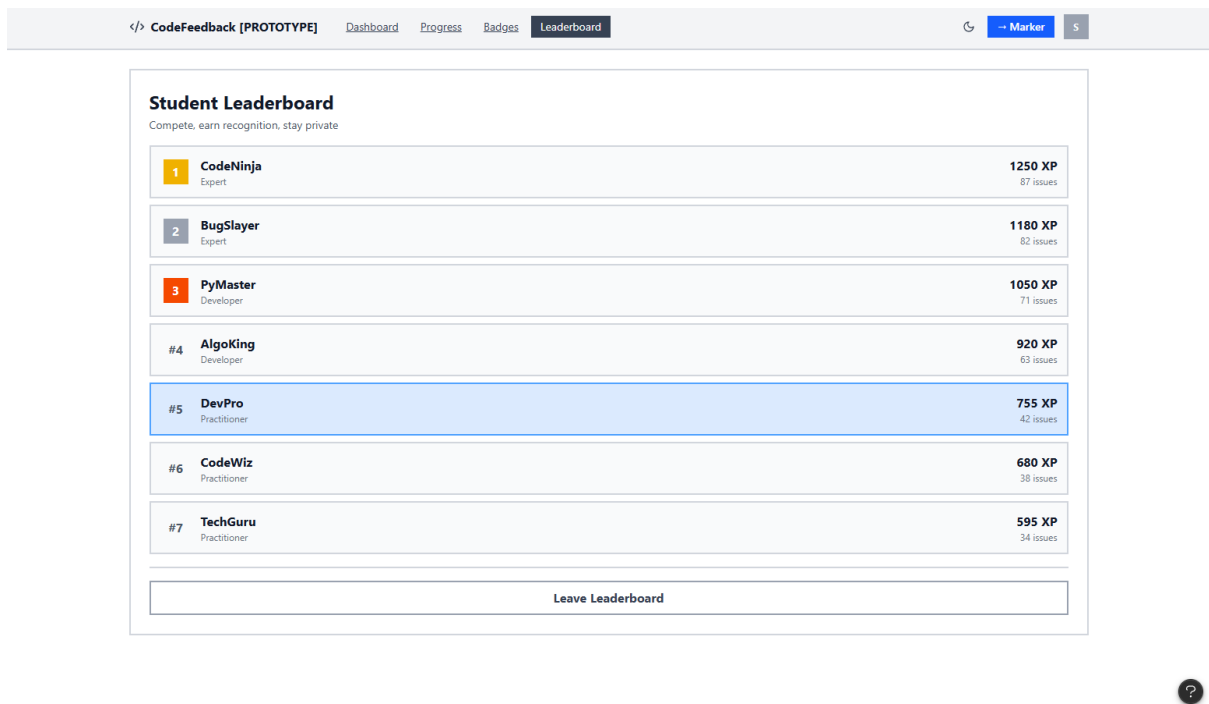


Figure 5.3: Student Leaderboard Interface Displaying Rankings, XP, and User Levels

5.2.1 Student Interface

The user interface design of Code Feedback Studio was developed through an iterative and user-centred approach, focusing on clarity, usability, and alignment with the needs of students and markers. Initial

design decisions were guided by the core objective of reducing cognitive load and ensuring that users can easily understand feedback, track progress, and navigate the system. As shown in **Figure 5.1**, the student dashboard was designed to provide a clear overview of course progress, highlighting completion status and key metrics in a simple and structured layout. This ensures that students can immediately understand their current standing without unnecessary complexity.

Building on this, the progress interface (**Figure 5.2**) was designed to provide deeper insights into performance through visual elements such as progress bars, XP tracking, and issue categorisation. The inclusion of metrics such as total issues, issues fixed, and fix rate was intended to make improvement measurable and encourage reflective learning. Multiple iterations of this interface were required to balance detail with readability, ensuring that the information remains informative without overwhelming the user.

The leaderboard interface (**Figure 5.3**) was introduced as part of the gamification strategy to enhance engagement and motivation. It was designed to display rankings, XP, and user levels in a clean and consistent format, while also maintaining user comfort through features such as optional participation and anonymised usernames. This reflects a careful design choice to support motivation without introducing excessive competitive pressure.

These interfaces were implemented using React and Tailwind CSS, allowing for a component-based and consistent design across the platform. The iterative refinement of these designs was influenced by usability considerations and feedback gathered during development, ensuring that the final implementation aligns with both functional requirements and user expectations. Overall, the design process demonstrates a balance between simplicity, functionality, and engagement, resulting in an interface that effectively supports the feedback and learning workflow.

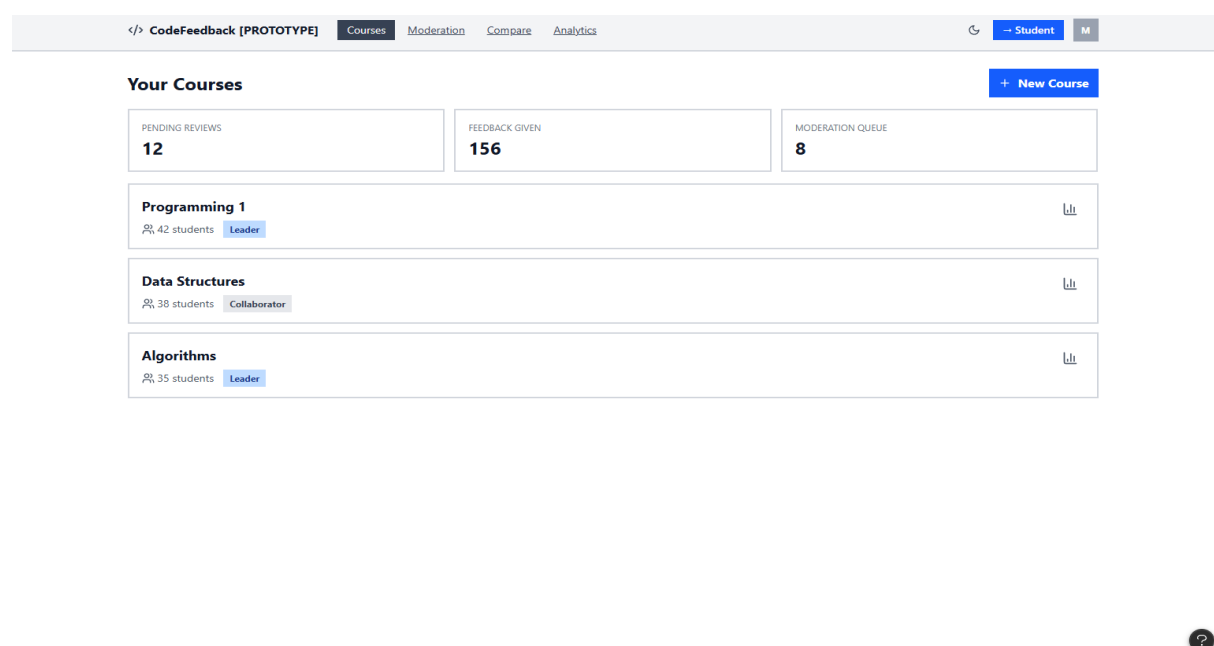


Figure 5.4: Marker Course Dashboard Interface

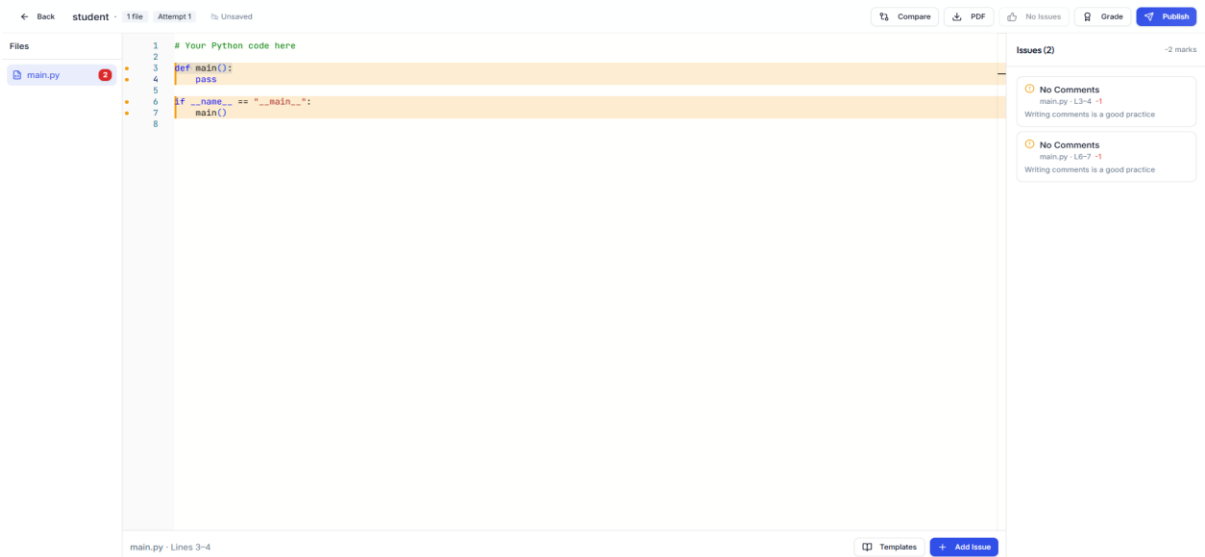


Figure 5.5: Inline Code Feedback Interface

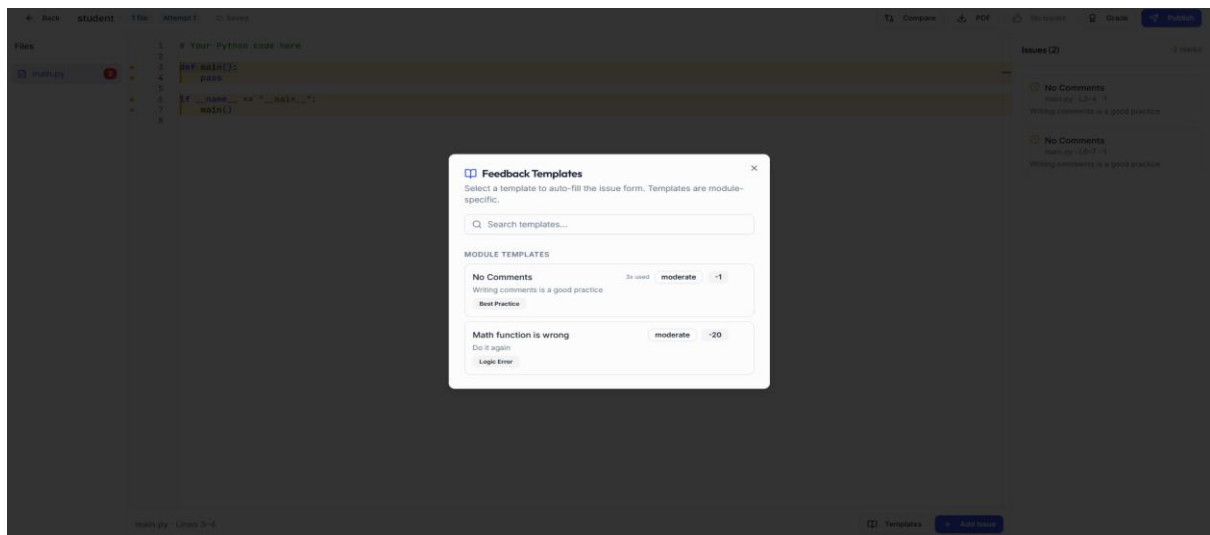


Figure 5.6: Feedback Template Selection Modal

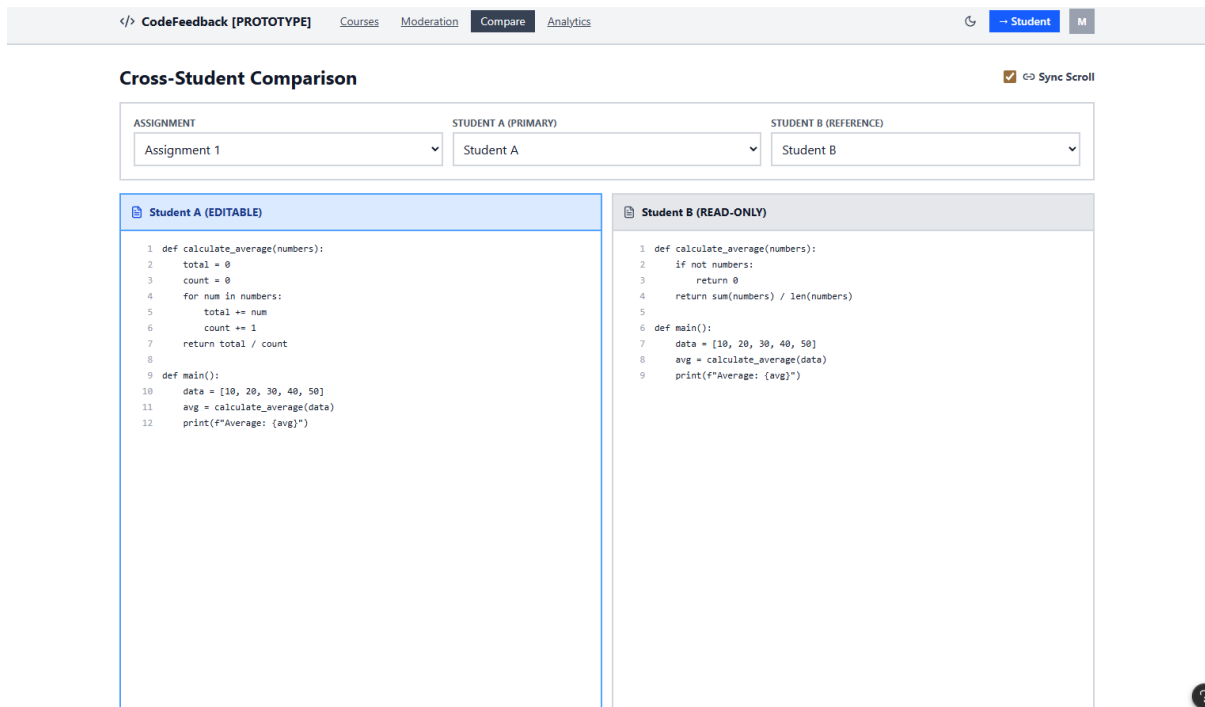


Figure 5.7: Cross-Student Code Comparison Interface

5.2.2 Marker Interface

The design of these interfaces was driven by a strong focus on usability, efficiency, and real-world marking workflows. Initially, low-fidelity layouts were conceptualised to identify how markers interact with submissions, particularly the need for quick navigation between courses, efficient feedback input, and clear visibility of issues. Based on these requirements, the marker dashboard (Figure 5.4) was designed using a card-based layout to provide a concise overview of workload, enabling quick access to courses and key metrics.

The inline feedback interface (Figure 5.5) was developed as the core feature of the system, ensuring that feedback is directly attached to specific lines of code. This design decision was influenced by the need to reduce the disconnect between comments and code, making feedback more actionable and easier for students to understand. To further improve efficiency, the feedback template modal (Figure 5.6) was introduced, allowing markers to reuse structured comments. This feature was iteratively refined to balance speed and flexibility, ensuring templates could be applied quickly without limiting customisation.

Additionally, the cross-student comparison interface (Figure 5.7) was designed to support consistency in marking. By enabling side-by-side viewing with synchronised scrolling, markers can compare different submissions effectively, helping maintain fairness across evaluations. Throughout development, these interfaces were implemented using React components and integrated with the Monaco Editor, ensuring both performance and a familiar coding environment. The final designs reflect an iterative process of refinement, where usability considerations, feedback workflows, and technical constraints were continuously balanced to produce an efficient and user-centred system.

5.3 Design Principles

The interface design of Code Feedback Studio focuses on making complex academic workflows clear and manageable. Given the volume of information presented—such as code, feedback, and progress—the

design prioritises simplicity and clarity. Key information is structured using consistent layouts, clear headings, and minimal visual clutter to reduce cognitive load and support task-focused interaction.

Consistency is maintained across the platform through repeated layouts, typography, and visual patterns. Elements such as dashboards and feedback views follow a uniform card-based structure, allowing users to navigate the system intuitively without relearning interactions. A clear visual hierarchy ensures that key actions, such as reviewing submissions or publishing feedback, remain prominent and accessible.

Accessibility considerations include readable typography, strong contrast, and the inclusion of dark mode to support extended use. However, accessibility was primarily addressed through design choices rather than formal testing, indicating an area for future improvement through WCAG-based evaluation and assistive technology support.

Finally, the design balances usability with engagement. Gamification features such as badges and leaderboards are included to enhance motivation but are visually controlled to avoid distraction. Similarly, tools like cross-student comparison support marking consistency while maintaining a focus on structured, rubric-based evaluation. Overall, the design aims to deliver a clear, efficient, and user-focused experience, with further evaluation required to assess its long-term impact.

Chapter 6: Implementation

6.1 Introduction to Implementation

Code Feedback Studio was developed using a full-stack architecture, where the frontend, backend, and database were designed as separate but interconnected components. This separation allowed each part of the system to be developed, tested, and maintained independently, improving overall system reliability and scalability.

An incremental and modular development approach was followed throughout the project. Core features such as user authentication, course management, and code submission were implemented first, as they form the foundation of the system. Once these were stable, more advanced features such as inline feedback, feedback templates, moderation, gamification, and PDF export were added. This approach reduced complexity, allowed continuous testing, and ensured that each feature could be validated before moving on to the next stage.

The implementation was guided by the requirements specification, ensuring that all functional and non-functional requirements were systematically addressed. Each requirement was mapped to specific backend APIs, database models, and frontend components. This ensured consistency between the system design and the final implementation, and made it easier to verify that all required functionality had been completed.

6.2 Class Diagram

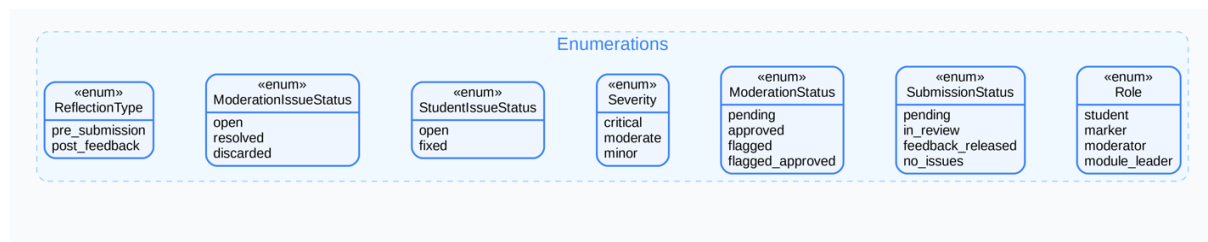


Figure 6.1: Enumeration Types Used in the System

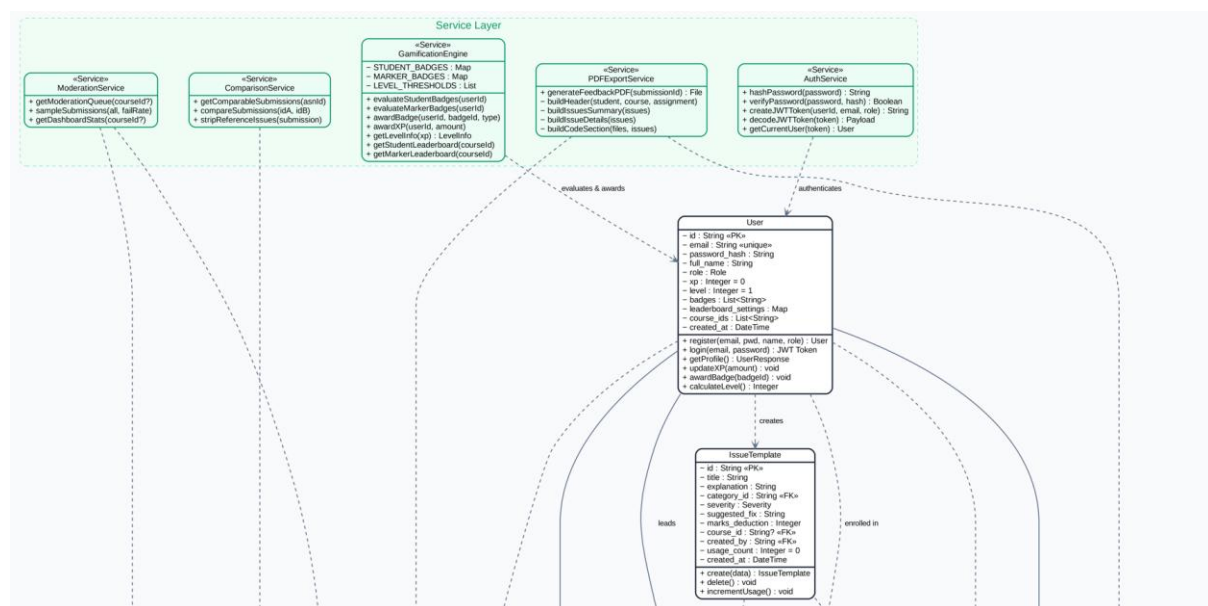


Figure 6.2: Service Layer and Core Class Relationships (Part 1)

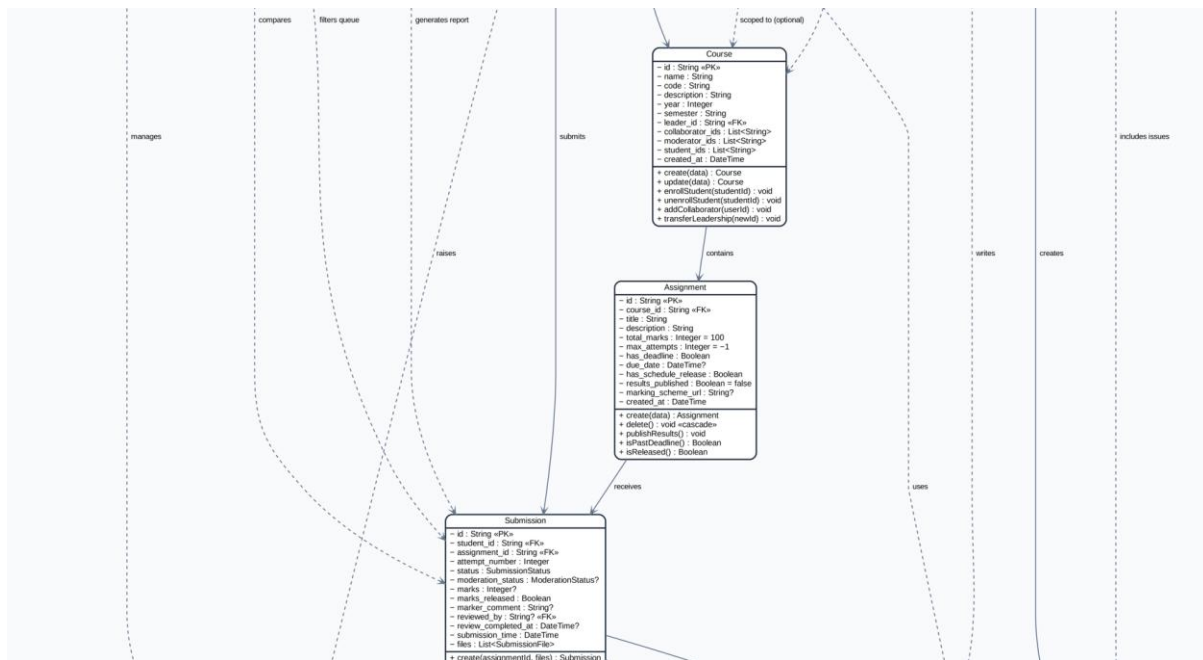


Figure 6.3: Core Domain Model – Course, Assignment, and Submission (Part 2)

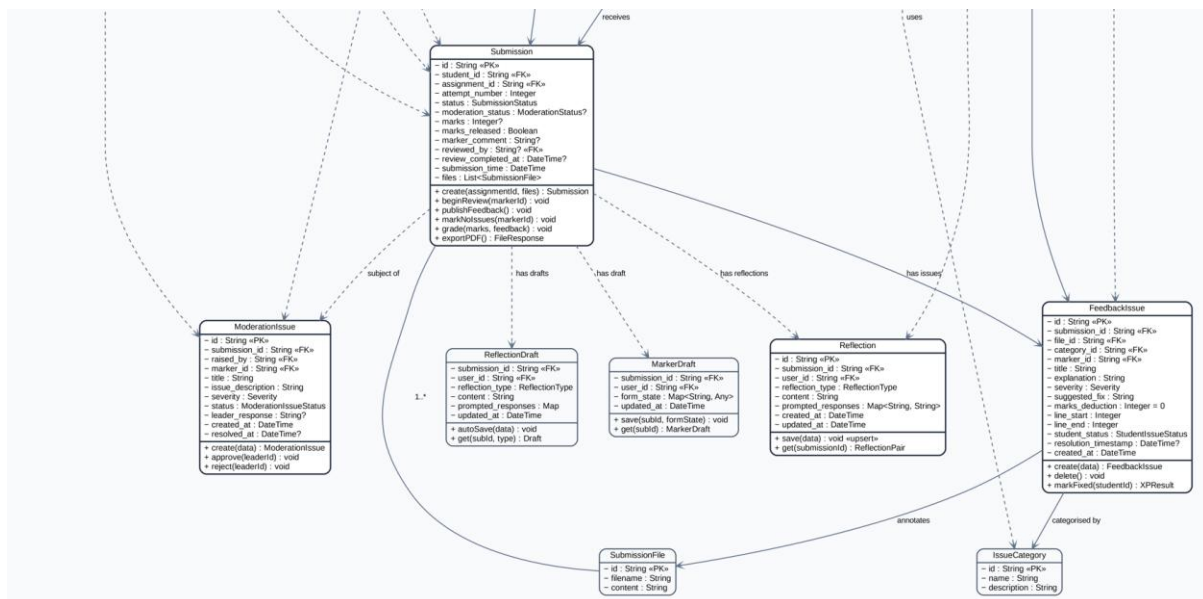


Figure 6.4: Feedback, Moderation, and Supporting Classes (Part 3)

The class diagram (Figures 6.1–6.4) provides a comprehensive view of the system architecture by illustrating the main classes in Code Feedback Studio and the relationships between them. Rather than presenting a single simplified diagram, the model is intentionally divided into multiple sections to improve readability and highlight different aspects of the system, including enumerations, service logic, core domain entities, and supporting components. This layered representation helps bridge the gap between conceptual design and actual implementation.

At the centre of the system is the User class (Figure 6.2), which represents all user roles, including students, markers, moderators, and module leaders. This unified structure avoids redundancy by managing all user types within a single model, while role-based behaviour is handled through the Role

enumeration (Figure 6.1). The inclusion of attributes such as XP, badges, and leaderboard statistics also reflects the integration of gamification directly into the core data model.

The Course and Assignment classes (Figure 6.3) define the academic structure of the system. A course acts as a container for multiple assignments and users, while each assignment encapsulates submission rules, deadlines, and marking configurations. This hierarchical design ensures that relationships between modules, coursework, and users are clearly defined and scalable.

The Submission class (Figures 6.3–6.4) acts as the central link between students, assignments, and the feedback process. It stores submission data, status transitions (e.g., pending, in review, feedback released), and marking outcomes. Its design reflects the need to manage both workflow states and associated data in a consistent and traceable manner.

The FeedbackIssue class (Figure 6.4) is a key component that enables structured, line-by-line feedback. By storing attributes such as severity, category, line range, and suggested fixes, it ensures that feedback is both standardised and context-specific. Supporting this are additional components such as IssueTemplate for reusable feedback, ModerationIssue for quality control, and Reflection classes for capturing student engagement with feedback.

The relationships across these classes demonstrate a deliberate design focused on modularity and separation of concerns. For example, courses contain assignments, assignments contain submissions, and submissions aggregate feedback issues and reflections. The service layer (Figure 6.2) further abstracts business logic such as authentication, comparison, PDF generation, and gamification, ensuring that core data models remain clean and maintainable.

Overall, the class diagram serves not just as a structural representation but as a reflection of key design decisions, including scalability, reusability, and clarity. By organising the system into interconnected but well-defined components, the implementation becomes easier to manage, extend, and maintain in a real-world academic environment.

6.3 Package Architecture

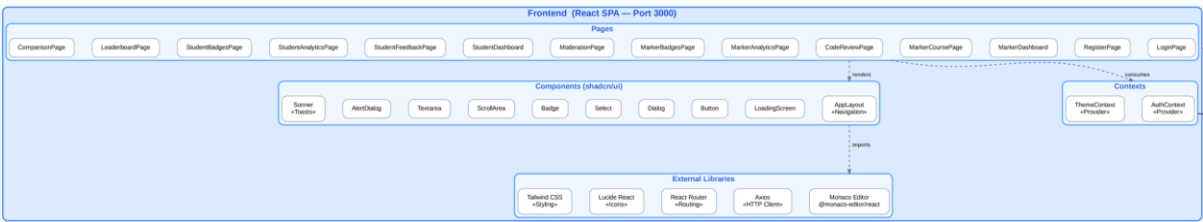


Figure 6.5: Frontend Architecture and Component Structure

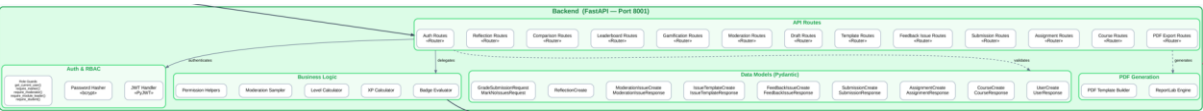


Figure 6.6: Backend Architecture and Service Layer Design

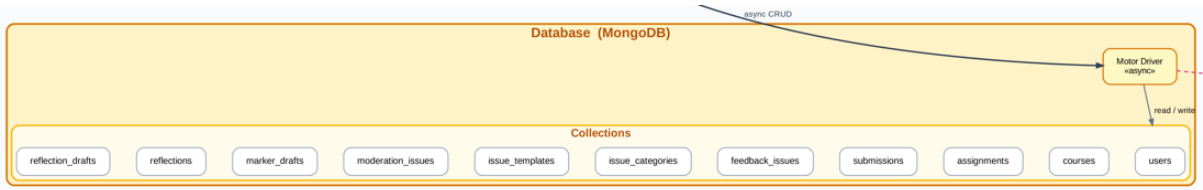


Figure 6.7: Database Schema and Collections (MongoDB)

The package diagram (Figures 6.5–6.7) illustrates how Code Feedback Studio is structured into three main layers—Frontend, Backend, and Database—and how these components interact to support the overall system. This layered architecture was intentionally adopted to ensure clear separation of concerns, making the system easier to develop, maintain, and scale over time.

The Frontend layer (Figure 6.5) represents all user-facing components and is built using React. It includes key pages such as dashboards, code review interfaces, comparison views, and analytics pages, along with reusable UI components like dialogs, buttons, and forms. Context providers are used to manage global states such as authentication and theme settings. This structure allows for modular and reusable interface design while ensuring smooth communication with the backend through API calls.

The Backend layer (Figure 6.6), implemented using FastAPI, handles all application logic and acts as the core processing unit of the system. It exposes API routes for functionalities such as authentication, submissions, feedback management, moderation, and analytics. In addition to routing, the backend includes dedicated service modules for handling complex operations such as gamification logic, submission comparison, and PDF generation. This separation between API routes and service logic ensures better organisation, testability, and scalability of the system.

The Database layer (Figure 6.7) is built using MongoDB and stores all system data in document-based collections. These include users, courses, assignments, submissions, feedback issues, templates, and reflections. The flexible schema design supports evolving requirements and allows new features to be integrated without major structural changes. The use of asynchronous database operations further enhances performance, particularly when handling multiple concurrent requests.

The interaction between these layers follows a clear and structured flow. The frontend sends requests to the backend via RESTful APIs, the backend processes these requests and applies the necessary business logic, and then communicates with the database to retrieve or update data. The processed results are then returned to the frontend, where they are rendered for the user. This design ensures that each layer has a well-defined responsibility, reducing coupling between components.

Overall, this package structure reflects a deliberate architectural decision to prioritise modularity, scalability, and maintainability. By separating the system into distinct layers while ensuring efficient communication between them, the design supports both current functionality and future extensibility in a real-world academic environment.

6.4 Implementation Details

Code Feedback Studio was implemented using a modern full-stack architecture with a clear separation between the frontend, backend, and database, improving maintainability and scalability.

The frontend was developed using React, enabling reusable components and consistent interfaces across dashboards and code review views. Tailwind CSS supported a clean and responsive design, while the Monaco Editor provided a robust code-viewing environment with syntax highlighting and line-level interaction.

The backend was built using FastAPI, leveraging asynchronous programming to handle multiple requests efficiently. It manages core functionalities such as authentication, submissions, feedback, moderation, and gamification. Data validation is handled using Pydantic models to ensure structured and reliable data flow.

MongoDB was used as the database, storing data in flexible document-based collections. Relationships are managed through a combination of references and embedded data, with the Motor driver enabling asynchronous database operations.

Communication between the frontend and backend is handled via RESTful APIs secured with JWT authentication and role-based access control. The system was developed incrementally, with features implemented and tested iteratively through manual and API testing to ensure reliability and stability.

6.5 Challenges and Solutions

During development, several technical and design challenges arose due to the system's complexity and multi-user nature.

A key challenge was maintaining consistency in feedback across different markers. Without standardisation, variations in style and detail could reduce the clarity of feedback for students. This was addressed through the introduction of structured feedback formats and reusable templates, enabling markers to deliver consistent, high-quality comments while also improving marking efficiency.

Another critical issue was preventing data loss during the marking process. Given the time-intensive nature of reviewing submissions, any loss of progress would significantly impact usability. To mitigate this risk, an auto-draft saving mechanism was implemented, periodically storing user input and ensuring that work could be recovered reliably.

Designing the data model also presented challenges, particularly in managing relationships between users, submissions, and feedback. This was resolved through a carefully structured schema using MongoDB, combining references and selective embedding to maintain data integrity while minimising query complexity.

Performance considerations became important when handling large submissions and concurrent users. This was addressed by leveraging asynchronous programming in FastAPI and optimising database operations, enabling efficient request handling and improved system responsiveness.

Finally, security was a fundamental concern due to the handling of sensitive academic data. This was addressed through JWT-based authentication and strict role-based access control, ensuring that all system actions are properly authorised and aligned with user permissions.

Overall, these challenges informed key architectural and design decisions, contributing to a system that is both robust and scalable.

Chapter 7: Development Methodology and Tools

The development of Code Feedback Studio followed an Agile-inspired approach, prioritising flexibility, iterative delivery, and continuous refinement. Rather than a rigid linear model such as Waterfall, the system was developed in incremental stages, allowing features to be progressively implemented, tested, and improved. This approach was well suited to evolving requirements and the complexity of integrating components such as submissions, feedback, moderation, and gamification. Agile methodologies are recognised for their ability to manage uncertainty and adapt to change ([Beck et al., 2001 \[20\]](#)). In this project, an Agile-inspired process enabled continuous validation and responsiveness to technical challenges and user feedback. However, it was a pragmatic adaptation rather than a formal implementation of frameworks such as Scrum or Kanban, which limited structure and formal evaluation.

7.1 Agile Development Approach

Development was organised around incremental feature delivery, with core functionalities implemented first, followed by more advanced features. Early stages focused on foundational components such as authentication and course management, while later iterations introduced features such as inline feedback, templates, moderation workflows, and PDF export.

Each cycle followed a simplified iterative process: feature design, implementation, testing, and integration. This approach supported early issue detection and reduced the risk of large-scale failures. For example, the auto-draft saving feature was introduced in response to a usability risk identified during marking.

However, the absence of formal sprint planning, backlog management, and retrospectives limited progress tracking and evaluation. While flexibility was achieved, a more structured Agile approach could have improved visibility, efficiency, and overall project management.

7.2 Software Tools and Technologies

The technology stack was selected to support the main engineering goals of Code Feedback Studio: interactive code review, secure multi-role access, scalable backend communication, flexible data storage, and a responsive user interface. Rather than selecting tools purely based on popularity, each technology was chosen according to how well it supported the system's functional and non-functional requirements. However, these choices also involved trade-offs in complexity, maintainability, and long-term scalability.

7.3 Frontend Technology Choices

React was chosen for the frontend because the platform required a highly interactive interface with reusable components, such as dashboards, feedback panels, code review pages, badge displays, and leaderboard views. Its component-based structure made it suitable for separating complex interface elements into manageable parts, improving maintainability and reducing duplication.

Tailwind CSS and shadcn/ui were used to support rapid development of a consistent and accessible interface. This was useful for maintaining a uniform visual style across student, marker, moderator, and module leader views. However, utility-first styling can make components harder to read as the interface grows, meaning future development would benefit from stronger component abstraction and style organisation.

The Monaco Editor was a key frontend decision because the core purpose of the system is code-based feedback. Unlike a basic text area, Monaco provides syntax highlighting, line selection, glyph margins, and a familiar coding experience similar to Visual Studio Code. This directly supports the project's

educational aim of attaching feedback to exact code locations. The trade-off is that Monaco increases frontend complexity and requires careful handling of editor state, file switching, and line-based annotations.

7.4 Backend Technology Choices

FastAPI was selected for the backend because it supports asynchronous request handling, automatic API documentation, and clear integration with Python data validation tools ([Ramírez, 2021 \[22\]](#)). This made it suitable for a multi-user platform where students, markers, moderators, and module leaders may perform different actions at the same time. It also supported a clean REST API structure for authentication, submissions, feedback issues, moderation, gamification, PDF export, and comparison workflows.

Pydantic was used to validate request and response data, reducing the risk of malformed inputs entering the system. This was especially important because the platform handles structured academic data, including submissions, feedback issues, marks, templates, and reflections. JWT authentication and bcrypt password hashing were used to provide secure, stateless access control. This supported the role-based access model while avoiding server-side session storage. This aligns with established principles of secure authentication and cryptographic protection in web systems ([Stallings, 2017 \[23\]](#)).

The main trade-off with FastAPI's asynchronous model is added implementation complexity. While async operations improve responsiveness, especially when communicating with MongoDB through Motor, they require careful error handling to avoid inconsistent states or difficult-to-debug issues.

7.5 Database and Data Management

MongoDB was chosen because the system contains flexible and evolving data structures, including multi-file submissions, feedback issues, templates, reflections, badges, draft states, and leaderboard settings. A document-based database made it easier to represent these nested structures without needing a large number of relational joins. This was particularly useful during development because the schema evolved as features such as moderation, gamification, and reflections were added.

Motor, the asynchronous MongoDB driver, was used to align database operations with FastAPI's async backend. This improved responsiveness by allowing the backend to handle database interactions without blocking other requests.

However, MongoDB also introduces important trade-offs. Unlike relational databases, it does not enforce strict relationships between entities such as users, courses, assignments, submissions, and feedback issues. This means data integrity must be maintained carefully through validation, application logic, and consistent use of identifiers. This reflects a known trade-off in NoSQL database design, where flexibility is gained at the cost of stronger relational consistency ([Sadalage and Fowler, 2012 \[24\]](#)). For a future institutional version of the system, a relational database such as PostgreSQL may provide stronger consistency guarantees, especially for assessment records and audit trails. Therefore, MongoDB was appropriate for rapid development and flexible feature expansion, but it may not be the strongest long-term choice for a production university deployment.

7.6 Development and Deployment Tools

Development was supported by Yarn and pip for dependency management, Axios for frontend-backend communication, and CRACO for custom frontend configuration. Axios was useful because the system required repeated authenticated API requests across many pages, including submissions, feedback issues, moderation actions, and leaderboard data.

For deployment, the frontend was designed for Vercel, the backend for Render, and the database for MongoDB Atlas. This separation reflected the layered architecture of the system and allowed each part to be deployed independently. It also made the project easier to test and demonstrate without requiring university infrastructure. This approach is consistent with modern cloud-based deployment practices, where application components are separated and deployed as scalable services ([Pahl, 2015 \[25\]](#)).

However, the deployment process was not fully automated. The lack of continuous integration and continuous deployment limited reliability because testing and deployment still depended partly on manual checks. A more mature version of the project should include GitHub Actions, automated backend tests, frontend regression tests, and possibly Docker-based deployment. This would reduce integration errors and make the system more suitable for production use.

7.7 Benefits and Limitations of the Approach

The combination of Agile-inspired development and modern software tools provided several advantages:

- **Flexibility:** The system could evolve as requirements changed
- **Reliability:** Incremental testing reduced system-wide failures
- **Scalability:** Asynchronous architecture supports multiple users
- **Maintainability:** Component-based design simplifies updates
- **Performance:** Efficient handling of concurrent requests

However, these benefits are accompanied by limitations. The lack of formal Agile structure reduced the ability to systematically evaluate development efficiency. Similarly, while the chosen technologies support scalability, the absence of large-scale testing means that real-world performance remains unverified. Furthermore, trade-offs in technology selection, such as using MongoDB over relational databases, introduce potential long-term challenges in data consistency.

7.8 Conclusion

Overall, the use of an Agile-inspired development approach combined with modern software tools played a significant role in the successful delivery of Code Feedback Studio. The iterative process enabled continuous refinement and responsiveness to emerging requirements, while the selected technologies supported the development of a functional, scalable, and maintainable system.

However, the approach also reveals areas for improvement. A more structured Agile methodology, earlier integration of automated testing, and deeper consideration of alternative technologies could have strengthened both the development process and the final system. These reflections highlight that while the chosen approach was effective within the scope of the project, further refinement is required to fully align with best practices in professional software engineering.

Chapter 8: Testing and Evaluation

Testing and evaluation formed a core part of the development of Code Feedback Studio. Due to the system's complexity supporting multiple user roles, interconnected workflows, and a wide range of features it was essential to ensure that the platform operates correctly, securely, and efficiently in all scenarios.

To achieve this, I followed a continuous and iterative testing approach, where testing was carried out alongside development rather than only at the end. After implementing each feature, it was immediately tested before moving forward. This approach helped identify issues early, reduce debugging complexity, and ensure that errors did not affect other parts of the system.

In addition to in-report explanations, I maintained a detailed testing record in a separate [.md file \[9\]](#) (located within /backend/tests/ and supporting documentation). This file includes all test cases, expected outputs, actual results, and iteration-wise improvements. It serves as formal evidence of the testing process and allows the reader to explore the testing in greater depth if required.

8.1 Overall Testing Approach

The testing strategy was designed to evaluate the system from both a technical and user perspective. Instead of relying on a single testing method, I used a combination of:

- Backend (API) testing
- Frontend (user interaction) testing
- Integration (end-to-end workflow) testing
- Manual exploratory testing

Each method focused on a different aspect of the system. Backend testing ensured correctness of logic and data handling, frontend testing ensured usability and interaction quality, integration testing verified that components work together correctly, and manual testing helped simulate real-world usage.

This layered approach ensured that the system was tested thoroughly at all levels, from individual components to complete workflows.

8.2 Backend Testing

Backend testing was carried out using pytest, where automated test cases were written to simulate API requests and verify responses. Each endpoint was tested under different conditions to ensure reliability.

For every feature, I tested:

- Valid inputs to confirm expected behaviour
- Invalid inputs to ensure proper error handling
- Edge cases to check robustness

For example, in authentication testing:

- Correct login credentials were verified to return a valid token
- Incorrect credentials were confirmed to return appropriate error responses

- Unauthorised requests were blocked correctly

Similarly, for core system features, I tested:

- Submission creation and retrieval
- Feedback issue creation, editing, and deletion
- Template usage and reuse
- Moderation workflows and status changes

All major components of the system including authentication, courses, assignments, submissions, feedback, templates, moderation, gamification, PDF export, and comparison features were covered in testing.

A full breakdown of backend tests, including detailed inputs and outputs, is available in the [.md testing file \[9\]](#) (located within /backend/tests/ and supporting documentation as well as test_reports, demonstrating comprehensive API coverage.

8.3 Frontend Testing and User Interaction

Frontend testing was performed by interacting with the system through the browser to simulate real user behaviour. The aim was to ensure that all features were accessible, intuitive, and functioning correctly.

Testing was carried out across different roles:

- Student: submitting code, viewing feedback, marking issues as fixed
- Marker: reviewing submissions, adding feedback, grading, publishing results
- Moderator: reviewing flagged issues and moderation decisions

During testing, I verified:

- Correct functionality of UI elements (buttons, forms, navigation)
- Accuracy of displayed data compared to backend responses
- Smooth transitions between pages and features

This ensured that the system is not only technically correct but also user-friendly and easy to navigate.

8.4 Integration Testing (End-to-End Workflows)

Integration testing focused on verifying that different parts of the system work together correctly. Instead of testing features individually, I tested complete workflows from start to finish.

A key workflow tested repeatedly was:

1. A student submits code
2. A marker reviews the submission and adds feedback
3. Feedback is published
4. The student views and interacts with feedback

Testing this process ensured that:

- Data flows correctly between frontend, backend, and database
- System states update properly (e.g., submission status changes)
- Users receive the correct outputs at each stage

This type of testing is essential because it reflects real-world usage scenarios.

8.5 Data Validation and Edge Case Testing

To ensure system robustness, I tested how the system handles incorrect or unexpected inputs.

This included:

- Invalid email formats during registration
- Missing or incomplete form data
- Incorrect values for feedback severity
- Invalid or unauthorised API requests

I also tested edge cases such as:

- Submissions after deadlines
- Duplicate leaderboard nicknames
- Accessing data belonging to another user
- Exporting non-existent submissions

These tests ensured that the system handles errors gracefully and remains stable under unexpected conditions.

8.6 Security Testing

Security testing was carried out to verify that authentication and authorisation mechanisms function correctly.

I tested scenarios such as:

- Accessing protected routes without authentication
- Users attempting to perform actions outside their role
- Ensuring sensitive data is only accessible to authorised users

JWT-based authentication and role-based access control were validated thoroughly to ensure that all actions are properly authorised and secure.

8.7 Performance Evaluation

The system's performance was evaluated by observing response times and behaviour under typical usage conditions. Most API requests were processed within approximately 50–200ms, indicating a generally responsive system during standard operation. More resource-intensive tasks, such as PDF generation, exhibited higher latency but remained within acceptable functional limits for the intended use case. The use of asynchronous programming in FastAPI, combined with optimised database queries, contributed to efficient handling of concurrent requests and reduced blocking operations.

However, this evaluation is limited in scope. Performance testing was conducted under relatively controlled conditions and did not simulate high user loads or stress scenarios. As a result, the system's scalability and behaviour under peak usage (e.g., large cohorts submitting or reviewing simultaneously) remain unverified. Additionally, the absence of formal benchmarking tools or load testing frameworks means that performance conclusions are based on observational data rather than rigorous quantitative analysis.

Therefore, while the results suggest that the system provides a responsive user experience under normal conditions, further testing particularly load and stress testing using tools such as JMeter or Locust would be necessary to validate performance reliability in real-world, large-scale deployments.

8.8 Regression Testing

Regression testing was performed after each major feature addition to ensure that existing functionality was not affected.

For example, after implementing templates and comparison features, I re-tested submission and feedback workflows to confirm that they continued to work correctly. This process helped maintain system stability and prevented new bugs from being introduced.

8.9 Testing Results and Evidence

The final testing results show that the system is stable and reliable. All major features were tested successfully, and no critical issues remained in the final version.

The accompanying [.md testing documentation \[9\]](#) (located in both the /backend/tests/ and /test_reports/ directories) provides:

- A complete list of test cases
- Expected and actual outputs
- Iteration-wise improvements
- Evidence of full system testing

This serves as clear proof that the system has been tested systematically and meets its requirements.

8.10 Critical Evaluation

While the testing process provided a reasonable level of confidence in the system's reliability and functionality, several limitations reduce the strength and generalisability of the evaluation.

Firstly, although backend testing was comprehensive and supported by automated test cases, frontend testing relied predominantly on manual interaction. While this approach enabled realistic simulation of user workflows, it lacks systematic coverage and is inherently prone to human oversight. As a result, subtle UI inconsistencies, edge-case failures, or regressions across updates may not have been fully captured. The absence of automated frontend testing frameworks such as Cypress or Selenium limits the repeatability and scalability of testing, particularly as the system evolves.

Secondly, performance evaluation was conducted only under normal operating conditions and did not include formal load or stress testing. Consequently, the system's ability to handle peak usage scenarios—such as large cohorts submitting assignments or multiple markers reviewing concurrently—remains uncertain. Without quantitative benchmarking or simulated high-concurrency environments, it is difficult to draw robust conclusions about system scalability or long-term performance stability.

In addition, testing was carried out primarily within a controlled development environment. This introduces further limitations, as real-world deployment conditions—such as variable network latency, differing device capabilities, and browser-specific behaviours—were not fully represented. These factors may significantly affect user experience and system reliability in practice, particularly in a university setting with diverse user environments.

More broadly, the evaluation focuses largely on functional correctness and perceived usability rather than measurable learning outcomes. While the system appears to improve the clarity and delivery of feedback, there is limited empirical evidence to demonstrate its impact on student performance or learning effectiveness. This represents a key gap between system functionality and educational impact.

To strengthen the evaluation and improve overall system robustness, several enhancements are recommended. These include the introduction of automated frontend testing to ensure consistent UI validation, implementation of load and stress testing using tools such as JMeter or Locust, and conducting usability studies with a larger and more diverse user base. Furthermore, integrating continuous integration and continuous deployment (CI/CD) pipelines would enable automated regression testing and improve development efficiency. Finally, future work should aim to incorporate empirical evaluation methods, such as controlled studies or performance comparisons, to better assess the system’s impact on learning outcomes.

8.11 User Feedback and Survey Analysis

Rate the dashboard	Rate the gamification	The app enhances the quality of feedback given to the students.	The app is easy to use and easy to learn.
5	3	5	5
5	5	5	5
3	2	5	5
5	5	4	5
4	5	5	5
5	4	5	5
5	5	5	5
3	4	5	5
5	5	5	5
5	5	5	5
5	5	4	5
5	5	5	5
5	5	5	5
5	4	5	4
5	5	5	5
5	5	5	5
5	5	5	5
5	5	5	5

Figure 8.1: Survey Results on Dashboard, Gamification, Feedback Quality, and Usability Ratings

What feature of the app did you find most useful when reviewing your feedback, and suggest me a new feature that I can add according to you?
The feedback bit really helped me understand my mistakes and it was easier for me to see and correct my mistakes. Leaderboard or able to see and share my profile will help the overall experience.
It looks very useful to me. I dont know how will University use this app tho
I really liked the concept the gamification could've been better. Maybe leaderboard will help with motivation to correct the mistakes among the students
this helps alot with the overall experience. more features are not required according to me
I think this app is perfect. Most of the features are there
Looks way better than the current uni's way of releasing feedback
Leaderboard would help by motivating ppl to fix the code
UI could be better
Looks perfect to me. If i were to add a feature i would add some sort of incentive to ppl to actually fix the mistakes
Looks good
exporting of feedback as a downloadable PDF could help
I think more features would ruin the simplicity of the platform and make it too complicated
Leaderboard
Some AI features would make it more upto date software
Looks good to me
The UI couldve been better. Rest looks good. Like could've had avatars
Really liked it. I dont think more features should be added or else it would get too complicated
Looks good

Figure 8.2: User Feedback on Most Useful Features and Suggested Improvements

Rate the dashboard	Rate the gamification	Rate creating the error template.	The app enhances the quality of feedback given to the students.	The app is easy to use and easy to learn.	The issues can be attached to a specific location in the code
5	5	5	5	5	5
5	5	5	4	5	4

Figure 8.3: Marker Evaluation of Templates, Feedback Quality, and Code Annotation Features

Rank the features [From the most useful feature to the least useful feature]
Highlighting Errors;Issue Template;Gamification;Dashboard;
Dashboard;Gamification;Issue Template;Highlighting Errors;

Figure 8.4: Ranking of Features Based on Perceived Usefulness

What features would you like to see added?
Release the feedback using university's email on the day when you publish feedback; Export the feedback to acceptable format of feedback for feedback.soton.ac.uk ; Automate testing the code, tests for students during submission and tests for markers to check the code after submission.
Co similarity among the codes; Abnormal submission (flaer een ai code)

Figure 8.5: Suggested Additional Features and System Enhancements from Users

To evaluate Code Feedback Studio, a user survey was conducted with 18 students and 2 markers. This provided valuable insight into real user experiences, which is essential in user-centred design for identifying usability issues and aligning systems with user needs ([Nielsen, 1994 \[13\]](#); [Brooke, 1996 \[14\]](#)). However, survey data remains subjective and does not directly measure learning outcomes.

Quantitative results (Figure 8.1) show consistently high ratings across key areas. Ease of use and feedback quality were almost universally rated 5, indicating strong agreement that the system is intuitive and effective. The dashboard also performed well, though with minor variation, suggesting small usability improvements may still be required. Gamification showed the greatest variation, reflecting mixed user perceptions of its importance.

A key finding is the strong positive response to inline feedback. As shown in Figure 8.3, the ability to attach issues directly to specific code locations was rated highly by nearly all participants, confirming it as a core strength of the system. This supports feedback theory, which highlights the importance of specific and contextual feedback ([Hattie and Timperley, 2007 \[2\]](#)). Feature rankings (Figure 8.4) further reinforce this, with highlighting errors and issue templates consistently ranked as most useful, while gamification and dashboards were less prioritised.

Qualitative responses (Figure 8.2 and Figure 8.5) provide additional insight. Users emphasised that embedded feedback improved understanding and made corrections easier. Suggested improvements included exporting feedback as PDF, integrating with academic systems, and adding automation features. At the same time, several users warned against overcomplicating the platform, highlighting the importance of maintaining simplicity ([Nielsen, 1994 \[13\]](#)).

The proposed leaderboard feature illustrates this trade-off. While some users viewed it as motivating, others raised concerns about unnecessary complexity and potential negative competition. Although gamification can enhance engagement ([Deterding et al., 2011 \[5\]](#)), the mixed responses suggest it must be implemented carefully.

Despite these insights, the evaluation has limitations. The small sample size reduces generalisability, and the clustering of high ratings limits differentiation between features. Additionally, the reliance on subjective feedback, absence of statistical analysis, and lack of a control group restrict the strength of conclusions.

Overall, the findings indicate that Code Feedback Studio is intuitive and effective, particularly in delivering contextual, code-linked feedback. However, further evaluation using larger samples and objective performance metrics is required to fully validate its impact on learning outcomes.

Chapter 9: Reflection on Product Delivery

9.1. Project Scope and Requirement Realization

The development of this system followed the MoSCoW method (Must, Should, Could, Won't) to decide which features were most important. As a final year project, the main goal was to build a reliable and well-functioning tool, while keeping the design manageable and ensuring it meets the security standards expected in a university setting.

9.1.1 Implementation of Core Functionalities (The "Must-Haves")

The core "Must-Have" features were built to make the feedback process smooth and effective. The main part of the system is an interactive code viewer, which allows markers to add detailed comments directly on the code. Security was also a key focus, with Role-Based Access Control (RBAC) used to manage what students and markers can access, and AES-256 encryption used to protect sensitive data, ensuring the system meets university security standards.

9.1.2 Evolution of the Gamification Engine

A notable deviation from the initial project proposal was the inclusion of the Leaderboard. Originally categorized as a "Won't-Have" to avoid potential negative competitive pressure, the project's direction shifted during the development phase.

- **The Pivot:** By integrating the leaderboard alongside the XP bars (Should) and Milestone Badges (Could), the system provides a comprehensive gamification suite.
- **Engineering Justification:** This was implemented using a non-intrusive ranking system designed to emphasise user progress and engagement rather than focusing only on raw scores. By rewarding improvement, consistency, and participation, the system creates a more supportive environment that motivates users to keep improving instead of discouraging lower-ranked users. This approach is supported by research such as [From Game Design Elements to Gamefulness: Defining Gamification \[5\]](#), which highlights that well-designed gamification should enhance user motivation and engagement rather than create negative competitive pressure.

9.2. Critical Evaluation of Data Policy and AI Integration

The most significant technical and ethical hurdle encountered was Should-Have #8: the implementation of AI-supported feedback suggestions.

9.2.1 The University of Southampton Data Policy Compliance

Despite the current industry trend toward Generative AI, I made the calculated decision to exclude AI integration. Under the [University of Southampton's Data Policy \[10\]](#), the institution must maintain absolute authority and oversight over student data.

- **The Black Box Risk:** Integrating third-party LLM APIs (like OpenAI) would have introduced a "Black Box" effect. Student source code intellectual property protected by university regulations would be transmitted to external servers, residing outside our governed infrastructure.
- **Sovereignty over Privacy:** To ensure the system remains a "closed-loop" environment, I prioritized data sovereignty. This ensures that no external entity has access to, or can train models on, the submissions of Southampton students

9.3. Technical Summary and MoSCoW Finality

The final system delivers the majority of the original backlog, with targeted adjustments made in response to policy and design considerations. Notably, the MoSCoW prioritisation evolved during development: the AI feedback feature was removed from the Should-Have category due to data policy constraints, while the leaderboard was promoted from Won't-Have to a delivered feature to strengthen user engagement.

Requirement Tier	Delivery Status	Engineering Note
Must-Have	100%	Full delivery of secure submission and feedback modules.
Should-Have	~88%	All features delivered except AI, which was vetoed on policy grounds.
Could-Have	100%	Accessibility themes and side-by-side diffing are fully functional.
Won't-Have	Shifted	The Leaderboard was promoted to "Delivered" to finalize the gamification stack.

9.4 Critical Evaluation

This project demonstrates a structured approach to software engineering, delivering both core functionality and selected extended features within scope. Features such as side-by-side comparison and the leaderboard improved usability and showcased flexibility. However, these introduced trade-offs; while the leaderboard can increase engagement, it may also create competitive pressure. This was mitigated through optional participation and anonymised nicknames.

A key limitation was feature prioritisation. The inclusion of gamification diverted focus from refining core functionalities, particularly feedback workflows and the marking interface. This highlights the challenge of balancing feature expansion with system maturity, where prioritising core stability would likely have produced a more polished outcome.

User feedback was incorporated relatively late. Although it informed UI improvements, earlier and iterative user involvement would have improved design decisions and reduced rework, emphasising the importance of continuous user-centred design.

From a technical perspective, the modular architecture was effective, but scalability and maintainability could be improved. The lack of early automated testing and continuous integration limited reliability, while clearer separation of concerns would support long-term development.

The exclusion of AI-assisted feedback ensured compliance with [data protection policies \[10\]](#) but limited automation. Exploring privacy-preserving alternatives could provide a better balance in future work. Additionally, the evaluation relied on a small, subjective sample, reducing the strength of conclusions.

Overall, the project highlights the importance of balancing feature ambition with refinement. Future improvements should focus on stronger prioritisation, earlier user involvement, and more rigorous testing and evaluation.

Chapter 10: Conclusion and Lessons Learned

This project set out to design and implement Code Feedback Studio, a web-based platform aimed at improving the quality, clarity, and consistency of feedback in university programming courses. While the system addresses key challenges identified in the literature, such as delayed feedback, lack of contextual clarity, and inconsistency across markers ([Nicol and Macfarlane-Dick, 2006 \[1\]](#); [Hattie and Timperley, 2007 \[2\]](#)), the most significant outcome lies in the learning gained throughout its development. A key takeaway from this project is the importance of translating theoretical principles into practical design. Implementing concepts such as clarity, timeliness, and actionable feedback within a real system required careful consideration of user workflows and interaction design. This reinforced that building effective educational technology is not solely a technical task but requires an understanding of how users learn and interact with systems. In future projects, I will approach system design with a stronger focus on aligning functionality with user behaviour and learning outcomes, rather than focusing only on feature implementation.

From a technical perspective, the project strengthened my understanding of full-stack development and the importance of modular and incremental design. Building the system in stages allowed for continuous testing and adaptation, reducing complexity and improving reliability. I also learned the importance of making informed trade-offs, such as prioritising core features over additional complexity and selecting technologies based on project requirements rather than preference. Going forward, I will apply this structured approach to development by breaking down complex systems into manageable components and validating each stage before progressing further.

Another important learning was the role of user feedback in shaping system design. The survey results demonstrated that users consistently prioritise clarity and usability over additional features. This led to decisions such as maintaining simplicity and implementing features like an optional leaderboard with nickname-based participation to balance engagement with user comfort. This experience highlighted the importance of user-centred design, and in future projects, I will incorporate user feedback earlier and more systematically to guide development decisions.

The project also provided valuable insight into the ethical and practical constraints of software development. The decision to exclude AI-assisted feedback due to data protection requirements under the [University of Southampton's policies \[10\]](#) emphasised that technical innovation must be balanced with legal and ethical considerations. This has shaped my understanding that system design is influenced not only by what is technically possible but also by what is responsible and compliant. In future work, I will be more mindful of data governance, privacy, and ethical implications when making design decisions.

Finally, the evaluation process highlighted the challenges of measuring system effectiveness. While the system performed reliably and user feedback was positive, the reliance on subjective measures and a small sample size limits the strength of the conclusions. This reinforced the importance of rigorous evaluation methods, including larger-scale testing, objective metrics, and comparative analysis. In future projects, I will prioritise more structured evaluation approaches to produce stronger and more reliable evidence of system impact.

Overall, this project has contributed significantly to my development as a software engineer. It has strengthened my ability to design user-focused systems, make informed technical decisions, and critically evaluate both the strengths and limitations of my work. While Code Feedback Studio provides a strong foundation for improving feedback processes, the experience has also highlighted that effective software development is an iterative process that requires continuous learning, evaluation, and refinement.

References

1. Nicol, D. J. and Macfarlane-Dick, D. (2006) *Formative assessment and self-regulated learning: A model and seven principles of good feedback practice*. Studies in Higher Education, 31(2), pp. 199–218. Available at: <https://doi.org/10.1080/03075070600572090>.
2. Hattie, J. and Timperley, H. (2007) *The power of feedback*. Review of Educational Research, 77(1), pp. 81–112. Available at: <https://doi.org/10.3102/003465430298487>
3. Gibbs, G. and Simpson, C. (2004) *Conditions under which assessment supports students' learning*. Learning and Teaching in Higher Education, 1, pp. 3–31.
4. Ithantola, P. et al. (2010) *Review of recent systems for automatic assessment of programming assignments*. In: Proceedings of the Koli Calling International Conference on Computing Education Research. Available at: <https://doi.org/10.1145/1930464.1930469>
5. Deterding, S. et al. (2011) *From game design elements to gamefulness: Defining gamification*. In: Proceedings of the 15th International Academic MindTrek Conference. Available at: <https://doi.org/10.1145/2181037.2181040>
6. Moodle (2024) *Moodle*. Available at: <https://moodle.org>
7. Blackboard (2024) *Blackboard*. Available at: <https://www.blackboard.com>
8. CodeGrade (2024) *CodeGrade*. Available at: <https://www.codegrade.com>
9. Shuklla, V. (2026) *Code Feedback Studio*. Available at: <https://github.com/VirathShuklla/CodeFeedbackStudio.git>
10. University of Southampton (2024) *Data protection policy*. Available at: <https://www.southampton.ac.uk/about/governance/regulations-policies/policies/data-protection>
11. Institute for the Future of Work (2023) *Inside the black box: Risk and uncertainty for AI in EU law*. Available at: <https://www.ifow.org/news-articles/inside-the-black-box-risk-and-uncertainty-for-ai-in-eu-law>
12. Scrum.org (2023) *Incremental delivery and principles of Agile Manifesto*. Available at: <https://www.scrum.org/resources/blog/incremental-delivery-and-principles-agile-manifesto>
13. Nielsen, J. (1994) *10 usability heuristics for user interface design*. Available at: <https://www.nngroup.com/articles/ten-usability-heuristics/>
14. Brooke, J. (1996) *System Usability Scale (SUS): A quick and dirty usability scale*. Available at: <https://digital.gov/topics/usability/>
15. Davis, F. D. (1989) *Perceived usefulness, perceived ease of use, and user acceptance of information technology*. MIS Quarterly, 13(3), pp. 319–340. Available at: <https://misq.umn.edu>
16. Norman, D. A. (2013) *The Design of Everyday Things*. Available at: <https://www.scirp.org/reference/referencespapers?referenceid=3905101>
17. Lazar, J., Feng, J. H. and Hochheiser, H. (2017) *Research methods in human-computer interaction*. Available at: <https://www.sciencedirect.com/book/9780128053904/research-methods-in-human-computer-interaction>

18. Tidwell, J. (2010) *Designing Interfaces: Patterns for Effective Interaction Design*. Available at: <https://www.oreilly.com/library/view/designing-interfaces-2nd/9781449379711/>
19. Budiu, R. (2020) *Usability of Dark Mode*. Available at: <https://www.nngroup.com/articles/dark-mode/>
20. Beck, K. et al. (2001) *Manifesto for Agile Software Development*. Available at: <https://agilemanifesto.org/>
21. Meta (2024) *React Documentation*. Available at: <https://react.dev/>
22. Ramírez, S. (2021) *FastAPI Documentation*. Available at: <https://fastapi.tiangolo.com/>
23. Stallings, W. (2017) *Cryptography and Network Security: Principles and Practice*. 7th edn. Pearson. (No direct free link available — textbook)
24. Sadalage, P. J. and Fowler, M. (2012) *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley. (No direct free link available — book)
25. Pahl, C. (2015) *Containerization and the PaaS Cloud*. IEEE Cloud Computing. Available at: <https://ieeexplore.ieee.org/document/7107397>